

GOTTFRIED WILHELM LEIBNIZ UNIVERSITÄT HANNOVER
FAKULTÄT FÜR ELEKTROTECHNIK UND INFORMATIK

Efficient Knowledge Graph Creation from RML-star Specifications

A thesis submitted in fulfillment of the requirements for the degree of
Bachelor of Computer Science

BY

Allwin Kuzhiyampurathu Malayil Biju

Matriculation number: 10035474

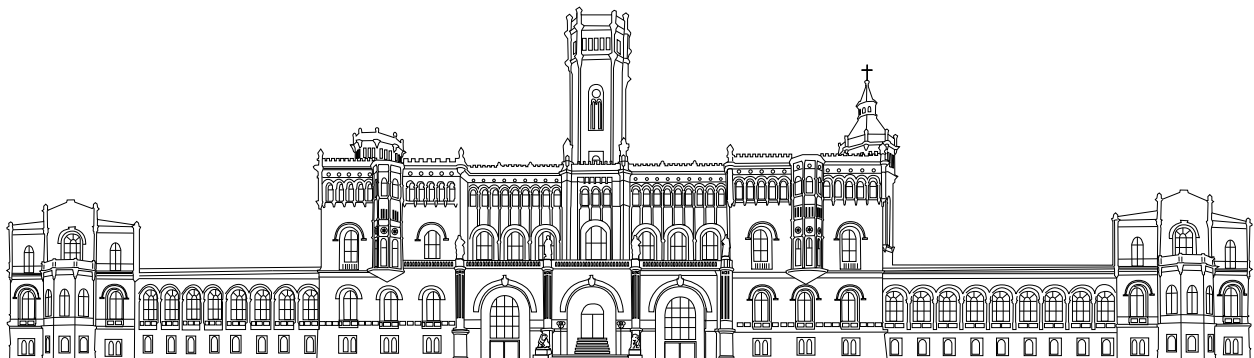
E-mail: allwin.kuzhiyampurathu.malayil.biju@stud.uni-hannover.de

First evaluator: Prof. Dr. Maria-Esther Vidal

Second evaluator: Prof. Dr. Sören Auer

Supervisor: M.Sc. Enrique Iglesias

May 15, 2026



Declaration of Authorship

I, Allwin Kuzhiyampurathu Malayil Biju, declare that this thesis titled, 'Efficient Knowledge Graph Creation from RML-star Specifications' and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.

Allwin Kuzhiyampurathu Malayil Biju

Signature: _____

Date: 15.05.2026

Acknowledgements

I would like to sincerely thank my supervisor, Enrique Iglesias, M.Sc., at L3S for his guidance, support, and valuable feedback throughout the course of this research.

I am also grateful to Prof. Maria-Esther Vidal and the TIB research group in Hannover for providing an excellent research environment and access to the department facilities and resources that made this work possible.

Their support and collaboration are greatly appreciated.

Abstract

RML-star extends RML with the ability to generate RDF-star statements, allowing triples themselves to be used as subjects or objects of other triples. This makes it possible to represent statement-level metadata directly, but it also introduces new execution challenges for materialization engines. In particular, quoted triples may become intermediate results that are reused by several later mappings, joined across different sources, or nested inside deeper quoted structures. A naive extension of an existing RML materializer may therefore appear efficient on simple mappings while still omitting valid output on more complex dependency patterns.

This thesis investigates robust RML-star materialization in an SDM-RDFizer-style execution model. It proposes SDM-RDFizer-Plus, a unified execution strategy that preserves the original Triples Map Planning and Triples Map Execution organization while extending it with star-aware planning and reuse mechanisms. The approach introduces explicit quoted dependency analysis, statement-level equivalence detection, dependency-aware cache lifetime management, and runtime structures for row-keyed and join-keyed quoted-result reuse.

The evaluation is correctness-first and memory-centered. It uses layered RML-star mapping families designed to stress same-source reuse, cross-source quoted joins, nested quoted contexts, and combined quoted subject/object patterns. The results show that the unified implementation preserves the expected output on the correctness gate and handles complex mapping patterns that expose limitations in the naive SDM-RDFizer implementation. Compared with Morph-KGC, SDM-RDFizer-Plus completes all level-2 and level-3 configurations and extends the feasible range into four-layer cases: it completes the mild-recursive control mapping and the L4C1 configuration at all evaluated scale points, and L4C2 at 100 000 rows, where Morph-KGC is already killed by the 28 GB memory limit. The two hardest patterns, L4C2 at 500 000 rows and L4C3, exceed the memory boundary for both engines.

Overall, the thesis shows that correct RML-star materialization requires quoted intermediate results to be treated as first-class reusable runtime objects rather than as ordinary emitted triples. The proposed unified strategy improves completeness and robustness for complex RML-star mappings, while identifying memory-efficient representation of retained quoted payloads as the main direction for future work.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Motivating Example	2
1.4	Contributions	3
1.5	Thesis Structure	4
2	Background	5
2.1	Data Silos and Heterogeneity	5
2.2	Classical Data Integration Systems	6
2.3	Knowledge Graphs	6
2.4	Resource Description Framework (RDF)	6
2.5	Mapping Languages: R2RML, RML, and RML-star	7
2.5.1	From RDF to RDF-star.	9
2.5.2	RML-star	9
2.6	SDM-RDFizer: An RML Knowledge Graph Engine	11
2.6.1	Overall Architecture	11
2.6.2	Planning-Phase Data Structures	11
2.6.3	Execution-Phase Data Structures	12
2.6.4	Physical Operators	12
2.7	Summary	12
3	Related Work	14
3.1	Knowledge Graph Construction	14
3.2	Mapping Engine Landscape	14
3.3	Engines Without Native RML-star Support	16
3.3.1	RMLMapper	16
3.3.2	CARML	17

3.3.3	RocketRML	19
3.3.4	Ontop	20
3.3.5	RMLStreamer	22
3.4	Engines Supporting RML-star / RDF-star	23
3.4.1	SDM-RDFizer (RML-star support)	23
3.4.2	Morph-KGC	26
3.5	Summary	27
4	Approach and Proposed Solution	29
4.1	Design Goals	29
4.2	Dependency Analysis	30
4.3	Statement Equivalence	30
4.4	Unified Reuse Layer	31
4.5	Cache Lifetime	31
4.6	Expected Benefits	32
5	Implementation	33
5.1	Implementation of the Unified Execution Strategy	33
5.2	Planning Phase: From Source Organization to Star-Aware Planning	35
5.2.1	Preserved Planning Structures	35
5.2.2	Unified Contribution in TMP	36
5.2.3	Summary of Implementation Changes in TMP	46
5.3	Execution Phase: From Triple Generation to Quoted-Result Reuse	46
5.3.1	Preserved Execution Structures and Operators	46
5.3.2	Unified Execution Data Structures	47
5.3.3	Base Operators in Unified Execution	52
5.4	Summary	64
6	Experimental Evaluation	65
6.1	Evaluation Goals	65
6.2	Experimental Setup	66
6.2.1	Data Sources and Dataset Sizes	66
6.2.2	Engines and Measurement Scope	66
6.2.3	Execution Environment	67
6.3	Layered Configuration Set	67
6.3.1	Two-Layer Family	67
6.3.2	Three-Layer Family	68
6.3.3	Intermediate Mild-Recursive Control	68
6.3.4	Four-Layer Family	69

6.4	Correctness-Oriented Results	69
6.4.1	The 3line Gate	69
6.4.2	Admissibility of Runtime and Memory Metrics	70
6.5	Scaling Results	71
6.5.1	Reading the Figures	71
6.5.2	Memory Behavior on Level 2 and Level 3	71
6.5.3	Deep Configurations and Memory-Limit Behavior	74
6.6	Discussion	76
7	Conclusions and Future Work	77
7.1	Conclusion	77
7.2	Future Work	79
	Bibliography	81

List of Figures

2.1	Resource Description Framework (RDF) triple representation from the running example: <code><http://ex.org/reading/R1> <http://example.org/fromSensor> <http://ex.org/sensor/S1></code> . The subject <code>ex:R1</code> is linked to the object <code>ex:S1</code> through the predicate <code>ex:fromSensor</code> .	6
5.1	Architecture of the unified RML-star implementation. The system extends the base RDFizer-style materialization workflow with planning-context construction, TM-level dependency analysis, quoted reuse, unified cache flushing.	34
5.2	Example of quoted dependency levels in the planning phase. A triples map without quoted dependencies starts at level 0. A triples map that consumes a level 0 quoted result is placed at level 1, and a later consumer is placed at level 2.	38
5.3	Statement-level equivalence between the non-asserted <code>RatedMovieStatementSupportTM</code> and the <code>pomRatedMovie</code> slice of the asserted <code>RatedMovieTM</code> . Both describe the same quoteable <code>ex:ratedMedium</code> payload, allowing the planner to reuse the materialized statement.	39
5.4	Planning views used to derive the final quoted execution order. The planner combines preserved OTML-style source ordering with quoted dependency levels to produce a level-aware execution schedule.	45
5.5	Equivalence view underlying the equivalence-aware reuse layer. The QET records that a non-asserted support triples map and a canonical owner-side predicate-object-map slice denote the same statement-level payload.	50
5.6	Same-row access to the equivalence cache. The cached statement-level payload is reused directly under the canonical owner-side equivalence identity for one exact row token.	51
5.7	Cross-source join access to the equivalence-aware reuse layer. The equivalence join table identifies canonical row tokens, which are then resolved through the row-keyed equivalence cache.	52

6.1	Peak RSS memory for the naive SDM-RDFizer implementation, Morph-KGC, and SDM-RDFizer-Plus on the 10000, 100000, and 500000 source runs. The memory axis is logarithmic base 10 in MB. Black dots mark memory-killed runs and diagonal stripes mark failed runs; these points are diagnostic and are not successful peak-memory measurements.	72
6.2	Runtime for the naive SDM-RDFizer implementation, Morph-KGC, and SDM-RDFizer-Plus on the 10000, 100000, and 500000 source runs. The time axis is logarithmic base 10 in seconds. For memory-killed runs, the shown value is time-to-kill; for failed runs, the shown value is not a completed materialization time.	73

List of Tables

1.1	Motivating-example results for the <code>mild_recursive_context</code> mapping.	3
5.1	Conceptual planning structures introduced by the unified TMP extension.	37
5.2	Concrete producer-centered QDT entry for a two-triples-map join example.	38
5.3	Conceptual QET entry for the <code>RatedMovieTM</code> support-map equivalence ex-ample.	40
5.4	Conceptual QFT entry for a producer-level flush boundary in the unified cache.	41
5.5	Conceptual QFT entry for the equivalence-group flush boundary with the canonical group ID <code>RatedMovieTM::pom::0</code> mentioned before in QET.	42
5.6	Conceptual unified-cache entries for <code>RatedMovieTM</code> .	48
5.7	Conceptual PJTT entries for quoted cross-source access from <code>movies.csv</code> to <code>RatedMovieTM</code> through <code>movieId</code> .	49
6.1	Growth factors for L4C1 from 10000 to 500000.	75

Acronyms

API Application Programming Interface

CLI Command-Line Interface

CSV Comma-Separated Values

DBMS Database Management System

DIS Data Integration System

DT Dictionary Table

FnO Function Ontology

IRI Internationalized Resource Identifier

JDBC Java Database Connectivity

JSON JavaScript Object Notation

KG Knowledge Graph

KGC Knowledge Graph Construction

OBDA Ontology-Based Data Access

OJM Object Join Map

ORM Object Reference Map

OTML Organised Triples Map List

OWL Web Ontology Language

PJTT Predicate Join Tuple Table

PL Predicate List

PTT Predicate Tuple Table

R2RML RDB to RDF Mapping Language

RDF Resource Description Framework

RDF-star RDF-star

RDFS RDF Schema

RML RDF Mapping Language

RML-star RML-star

RSS Resident Set Size

SOM Simple Object Map

SPARQL SPARQL Protocol and RDF Query Language

SQL Structured Query Language

TME Triples Map Execution

TMP Triples Map Planning

URI Uniform Resource Identifier

VKG Virtual Knowledge Graph

XML eXtensible Markup Language

Chapter 1

Introduction

Knowledge Graphs (KGs) provide a structured way to integrate data from different sources using a shared conceptual model. In **RDF**, this information is normally represented as subject–predicate–object triples. This representation is simple and widely used, but many applications also need to describe the statements themselves: for example, the source of a statement, the context in which it was observed, or an annotation attached to a particular assertion. **RDF-star (RDF-star)** addresses this need by allowing triples to be quoted and used inside other triples.

RML-star (RML-star) brings this idea into declarative knowledge graph construction. It extends **RDF Mapping Language (RML)** mappings so that generated statements can be quoted and reused as subjects or objects of other statements. This is useful for compact statement-level metadata, but it also changes the execution problem. In ordinary **RML** materialization, a generated triple is often only final output. In **RML-star** materialization, the same generated statement may also become an intermediate result that later mappings must reuse, join with other sources, or embed inside deeper quoted structures.

1.1 Motivation

The motivation for this thesis is that efficient **RDF** materialization does not automatically imply robust **RML-star** materialization. A naive extension of an **RDF** materializer can appear fast and memory-efficient on simple mappings, but still omit valid output when quoted statements are reused across several dependent rules. This can happen when quoted intermediate results are reconstructed independently, when their lifetime is not controlled explicitly, or when duplicate-suppression structures are reused in a way that interferes with quoted-result availability.

The central challenge is therefore not only to generate **RDF-star** syntax. The engine must manage quoted statements as execution objects. It must know which triples maps produce reusable quoted results, which later mappings consume them, which joins make them reachable, and when they can safely be released. Without this information, a materializer may either use too much memory by retaining intermediate results too broadly, or lose correctness by releasing or suppressing results that are still needed.

1.2 Problem Statement

Let $\mathcal{DIS} = (\mathcal{O}, \mathcal{S}, \mathcal{M})$ denote a declarative integration system, where $\mathcal{O}$ is the target ontology, $\mathcal{S}$ is a set of source instances, and $\mathcal{M}$ is a set of **RML-star** mappings. The goal is to construct a **KG** $\mathcal{K}$ that represents the source data according to the target ontology while preserving both asserted triples and quoted statement-level structures.

The problem addressed in this thesis is to design an execution strategy for integrating data in $\mathcal{S}$ as instance of $\mathcal{O}$ preserving mapping rules specified in **RML-star**. In this setting, generated statements may have a dual role: they may be final graph output, but they may also be quoted intermediate payloads required by later triples maps. A correct engine must therefore keep reusable quoted payloads available independently of final graph duplicate suppression, support same-source and cross-source access paths, and avoid unnecessary rematerialization.

This thesis addresses the problem by extending SDM-RDFizer with a unified execution strategy, called SDM-RDFizer-Plus. The strategy preserves the existing **Triples Map Planning (TMP)** and **Triples Map Execution (TME)** structure, but adds star-aware dependency analysis, managed quoted-result reuse, and dependency-aware cache cleanup.

1.3 Motivating Example

The motivating example is the `mild_recursive_context` mapping from the benchmark family. It is derived from the MovieLens source family and uses **Comma-Separated Values (CSV)** sources such as ratings and users. The mapping represents a common **RDF-star** pattern: a base statement is generated from one source, quoted and annotated, enriched with contextual information from another source, and then combined again in a higher-level statement.

This example is intentionally smaller than the deepest benchmark cases, but it already exposes the main execution difficulty. The same quoted statement is not

Table 1.1: Motivating-example results for the `mild_recursive_context` mapping.

Engine	Size	Time	Peak Resident Set Size (RSS)
SDM-RDFizer-Plus	100000	164.729 s	1953.75 MB
	500000	1417.853 s	11972.94 MB
Morph-KGC	100000	130.647 s	23268.75 MB
	500000	160.965 s	killed at 28 GB limit

consumed only once. It is reused by later mappings, participates in a cross-source join, and becomes part of another quoted context. A materializer therefore has to preserve row consistency, join correctness, and duplicate suppression while still making the intermediate quoted payload available to all dependent rules.

The table illustrates why memory behavior is central to the motivating example. At 100000, both engines complete, but Morph-KGC requires much more peak memory than SDM-RDFizer-Plus. At 500000, SDM-RDFizer-Plus still completes, while the Morph-KGC run reaches the configured 28 GB memory limit. The Morph-KGC time at this scale is therefore not a completed runtime measurement; it only indicates how quickly the run reached the memory boundary. The example therefore motivates the main thesis direction: quoted intermediate results must be reused deliberately, but their memory cost must also be controlled.

1.4 Contributions

This thesis makes four main contributions:

- it identifies correctness and robustness limitations that arise when naive SDM-RDFizer-style execution is applied to layered `RML-star` mappings;
- it proposes a unified execution strategy that makes quoted-result producers, consumers, equivalence relationships, and cache lifetimes explicit;
- it implements this strategy in SDM-RDFizer-Plus by separating reusable quoted payload retention from duplicate-suppressed final graph emission;
- it evaluates SDM-RDFizer-Plus against the naive SDM-RDFizer implementation and Morph-KGC on layered benchmark configurations, with correctness as the first admissibility condition for interpreting runtime and memory.

1.5 Thesis Structure

The remainder of the thesis is organized as follows. The background chapter introduces [RDF](#), [RDF-star](#), [RML](#), and [RML-star](#). The related work chapter discusses existing materialization engines and the limitations that motivate this thesis. Chapter [4](#) presents the design of the unified execution strategy. The implementation chapter describes how the strategy is integrated into the SDM-RDFizer planning and execution workflow. The evaluation chapter studies correctness, memory behavior, runtime behavior, and failure modes. The conclusion summarizes the findings and outlines future work, especially memory reduction for retained quoted payloads.

Chapter 2

Background

This chapter provides a concise overview of the core concepts and frameworks underpinning knowledge-graph-oriented data integration. Section 2.1 describes the challenges posed by data silos and heterogeneity. Section 2.2 introduces the concept of a classical Data Integration System (DIS). Section 2.3 introduces KGs. Section 2.4 presents the RDF. Section 2.5 introduces the mapping languages RDB to RDF Mapping Language (R2RML) and RML. Section 2.5.1 explains RDF-star and RML-star as extensions for statement-level annotation. Section 2.6 then introduces SDM-RDFizer as the baseline RML engine model relevant to this thesis. Finally, Section 2.7 summarises the chapter.

2.1 Data Silos and Heterogeneity

Modern information ecosystems are fragmented across relational databases, CSV files, JavaScript Object Notation (JSON) documents, eXtensible Markup Language (XML) feeds, and Application Programming Interfaces (APIs). This diversity produces *data silos* that impede interoperability, reuse, and integrated analysis. Beyond format heterogeneity, differences in schemas, identifiers, and vocabularies further complicate unified access and integration [5, 26]. Best-practice guidance for publishing and managing data on the web underscores the need for shared semantics and machine-actionable descriptions to improve discoverability and reuse [46].

2.2 Classical Data Integration Systems

Traditional **DIS** architectures rely on a *global schema*, *mappings* from sources to that schema, and *mediators* that rewrite queries against the sources [7, 17, 26]. While principled, these approaches often face rigidity when faced with evolving, semi-structured, and multi-format data. Maintaining mappings at scale can be costly, and performance degrades as sources and schemas proliferate.

2.3 Knowledge Graphs

A **KG** can be formally represented as a *directed edge-labeled graph* $G = (V, E, L)$, where $V \subseteq \text{Con}$ denotes the set of nodes, $L \subseteq \text{Con}$ the set of labels, and $E \subseteq V \times L \times V$ the set of labeled edges [45]. This structure captures entities and their semantic relationships, enabling the representation of complex interlinked information.

KGs provide a semantic abstraction that models data as entities and relationships, enabling cross-domain linking and supporting reasoning and complex queries [14, 21]. By adopting global identifiers (**Internationalized Resource Identifiers (IRIs)**) and shared vocabularies, **KGs** mitigate schema and identifier mismatches and create a uniform space for integration and analytics.

2.4 Resource Description Framework (RDF)

The **RDF** is a *flexible data model* used to represent information about resources on the web [45]. **RDF** expresses data as triples (**subject**, **predicate**, **object**), forming a *directed labeled graph* in which resources are identified by **Uniform Resource Identifiers (URIs)**.

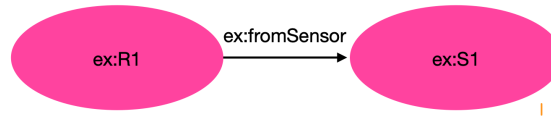


Figure 2.1: **RDF** triple representation from the running example: `<http://ex.org/reading/R1>` `<http://example.org/fromSensor>` `<http://ex.org/sensor/S1>`. The subject `ex:R1` is linked to the object `ex:S1` through the predicate `ex:fromSensor`.

Each resource is described through its properties and relationships with other resources, forming an interconnected graph-like structure. It offers multiple seri-

alisations (e.g., Turtle, RDF/XML, JSON-LD) and a well-defined abstract syntax for interoperability [27, 48]. The triple-centric model underpins many KG construction and publishing practices, and SPARQL Protocol and RDF Query Language (SPARQL) provides a standard query language over RDF graphs.

2.5 Mapping Languages: R2RML, RML, and RML-star

KG construction requires explicit rules that describe how source data is transformed into RDF. This is particularly important when the input comes from heterogeneous sources such as relational databases, CSV files, JSON documents, or XML files, which often differ in structure, naming conventions, and identifier schemes [44]. A common way to describe this setting is as a tuple $\langle O, S, M \rangle$, where S is a set of data sources, O is an ontology that defines the target vocabulary, and M is a set of mappings that connects values from the sources in S to the classes and properties in O . The ontology provides the global conceptual view, while the mappings specify how concrete source records are translated into RDF resources and relationships.

A mapping language provides these transformation rules declaratively. Instead of describing *how* an engine should procedurally process the source data, it specifies *which* RDF triples should be generated from which source values. This makes mappings easier to read, maintain, and reuse across systems.

R2RML [47] is the original declarative mapping language for generating RDF from relational databases. Its central construct is the *Triples Map*, which defines how one logical input record gives rise to RDF triples. Each Triples Map typically consists of a logical source, a subject map, and one or more predicate-object maps. RML [13] preserves this same structure, but generalises it beyond relational data so that the same mapping model can be used over CSV, JSON, XML, and other heterogeneous formats.

A simple RML example.

Consider the following CSV source:

id	name	age
1	Alice	23
2	Bob	31

A simple RML mapping for this source can be decomposed into the following components.

Logical Source

```
rml:logicalSource [  
  rml:source "persons.csv" ;  
  rml:referenceFormulation ql:CSV  
]
```

Subject Map

```
rml:subjectMap [  
  rml:template "http://example.org/person/{id}" ;  
  rr:termType rr:IRI  
]
```

Predicate-Object Map 1

```
rml:predicateObjectMap [  
  rml:predicate ex:name ;  
  rml:objectMap [ rml:reference "name" ]  
]
```

Predicate-Object Map 2

```
rml:predicateObjectMap [  
  rml:predicate ex:age ;  
  rml:objectMap [ rml:reference "age" ]  
]
```

Together, these components form one Triples Map. The logical source identifies the input data and the reference formulation used to access it. The subject map defines how an **RDF** subject is generated for each source row. Each predicate-object map then attaches one predicate-object pair to that subject. For the first row, the mapping produces **RDF** such as:

```
<http://example.org/person/1> ex:name "Alice" .
```

```
<http://example.org/person/1> ex:age "23" .
```

This example illustrates the basic purpose of **RML**: it declaratively specifies how source records are turned into **RDF** subjects, predicates, and objects. **R2RML** pro-

vides this model for relational sources, while [RML](#) extends the same model to heterogeneous data formats [\[47\]](#), [\[13\]](#).

2.5.1 From RDF to RDF-star.

Many applications require metadata *about* statements themselves, such as provenance, temporal validity, or confidence. Standard [RDF](#) can represent such metadata only indirectly, typically by introducing additional nodes or reification structures. [RDF-star](#) extends [RDF](#) with quoted triples, written as «s p o», so that statements can themselves appear in subject or object position [\[18\]](#). For example, the statement

```
«:bob :claims « :alice :birthYear 1996 » »
```

states that Bob claims that Alice was born in 1996. [RDF-star](#) therefore provides a more compact and natural way to represent statement-level metadata.

2.5.2 RML-star

[RML-star](#) extends [RML](#) so that mappings can generate [RDF-star](#) graphs directly from heterogeneous sources [\[11\]](#). In this formulation, `rml:AssertedTriplesMap` and `rml:NonAssertedTriplesMap` distinguish whether a triples map produces top-level [RDF](#) output or only a reusable quoted result. The former is materialized into the KG, whereas the latter is retained for quoted reuse without being asserted independently. Structurally, however, a `rml:NonAssertedTriplesMap` follows the same mapping pattern as an asserted triples map, that is, it may still define a logical source, a subject map, and predicate-object maps. Moreover, `rml:quotedTriplesMap` allows a subject map or object map to refer to the triples generated by another triples map, thereby using that quoted statement as the subject or object of a new triple.

To illustrate this, consider the following source fragment:

PERSON	BIRTHYEAR	CLAIMER	CONFIDENCE
alice	1996	bob	0.9
charlie	2002	daniel	0.3

A first inner mapping can generate the quoted statement about a person’s birth year.

Non-asserted triples map

```
<#RatedMovieIsolatedTM>
  a rml:NonAssertedTriplesMap ;
  rml:logicalSource [
    rml:source "data/ratings.csv" ;
    rml:referenceFormulation ql:CSV
  ] ;
  rml:subjectMap [
    rml:template "http://example.org/rating/{userId}/{movieId}" ;
    rr:termType rr:IRI
  ] ;
  rml:predicateObjectMap [
    rml:predicate ex:ratedMovie ;
    rml:objectMap [
      rml:template "http://example.org/movie/{movieId}" ;
      rr:termType rr:IRI
    ]
  ] .
```

This triples map produces a reusable quoted result, but because it is declared as a `rml:NonAssertedTriplesMap`, it is not emitted independently as an asserted top-level triple.

A second mapping can then use that quoted statement in subject position.

Quoted triple used in a subject map

```
<#RatingAuditTM>
  a rml:AssertedTriplesMap ;
  rml:logicalSource [
    rml:source "data/ratings.csv" ;
    rml:referenceFormulation ql:CSV
  ] ;
  rml:subjectMap [
    rml:quotedTriplesMap <#RatedMovieIsolatedTM>
  ] ;
  rml:predicateObjectMap [
    rml:predicate ex:ratingAudit ;
    rml:objectMap [
      rml:template "http://example.org/audit/{timestamp}" ;
      rr:termType rr:IRI
    ]
  ] .
```

RDF-star output is produced in this pattern, where the generated quoted statement is itself described by another triple. The same mechanism can also be used in object position. Thus, the key difference from standard **RML** is that subject and object positions are no longer restricted to ordinary **RDF** terms such as **IRIs**, blank nodes, or literals. Instead, they may refer to quoted triples generated by other triples

maps through `rml:quotedTriplesMap`.

2.6 SDM-RDFizer: An RML Knowledge Graph Engine

`R2RML` and `RML` shift the responsibility of efficient execution to the engine that interprets the mappings. Among the engines developed to this end, *SDM-RDFizer* has emerged as a state-of-the-art solution for large-scale `KG` generation [23]. In this thesis, *SDM-RDFizer* serves as the *baseline engine model* on top of which the final solution is developed. Accordingly, this section summarizes the baseline implementation — including its planning structures, runtime tables, and physical operators — so that the later unified solution can be understood as an extension of this baseline rather than as a completely separate engine.

2.6.1 Overall Architecture

SDM-RDFizer adopts a two-phase execution model: (i) the `TMP` phase, where the mappings are analysed and organised into efficient execution plans, and (ii) the `TME` phase, where data sources are scanned and triples are generated according to the plan. This separation allows the baseline *SDM-RDFizer* runtime to exploit structural properties of the mappings and to reuse intermediate results, improving the runtime efficiency. This work contribution preserves this overall execution skeleton, while refining how quoted-triple results are planned, cached, and reused for `RDF-star`-intensive workloads.

2.6.2 Planning-Phase Data Structures

During the `TMP` phase, *SDM-RDFizer* constructs specialised structures:

- **Organised Triples Map List (OTML)** groups all Triples Maps according to their data source, enabling each source to be scanned only once.
- **Predicate Lists (PLs)** cluster Triples Maps that produce triples with the same predicate, which helps reduce duplicate generation and informs the order of execution.

These structures anticipate join requirements and avoid redundant access to input files.

2.6.3 Execution-Phase Data Structures

At runtime, SDM-RDFizer relies on compact tables to manage and de-duplicate triples:

- The **Dictionary Table (DT)** encodes **IRIs** and literals into integers, reducing memory footprint and accelerating comparisons.
- The **Predicate Tuple Table (PTT)** records subject–object pairs per predicate, ensuring that duplicate triples are not materialised.
- The **Predicate Join Tuple Table (PJTT)** caches partial join results across Triples Maps, avoiding recomputation when multiple maps reuse the same join conditions.

Together, these tables allow SDM-RDFizer to scale to millions of triples without prohibitive memory overhead.

2.6.4 Physical Operators

The engine implements a set of operators corresponding to **RML**'s object map semantics:

- **Simple Object Map (SOM)**: directly generates triples from source values without joins.
- **Object Reference Map (ORM)**: resolves references between Triples Maps within the same source.
- **Object Join Map (OJM)**: executes joins between Triples Maps over different sources using **PJTT**.

Each operator produces candidate triples that are subsequently validated and de-duplicated against the Dictionary and Predicate Tuple Tables before serialization into the chosen **RDF** format.

2.7 Summary

This chapter has presented the foundational concepts underlying knowledge-graph-oriented data integration. It began by motivating the need to address data silos and heterogeneity through semantic abstractions such as **KGs**. The **RDF** was introduced

as a flexible model for representing graph-structured data. Declarative mapping languages, including [R2RML](#) and [RML](#), were presented as mechanisms for transforming heterogeneous data sources into [RDF](#). The chapter then discussed [RDF-star](#) and [RML-star](#) as extensions that enable statement-level annotations and highlighted the new execution challenges they introduce. Finally, it described the *baseline* architecture of SDM-RDFizer, which serves as the foundation for this thesis, including the data structures and operators used prior to the proposed star-aware optimizations. This background establishes the basis for the subsequent chapters, in which the unified solution is presented as an extension of the baseline rather than as a replacement for it.

Chapter 3

Related Work

3.1 Knowledge Graph Construction

The construction of [KGs](#) has been extensively studied in the literature. A comprehensive survey of [KG](#) creation, enrichment, and reasoning techniques is presented by Hogan et al. [\[21\]](#), who categorise [KG](#) construction pipelines into extraction, integration, and curation processes. Paulheim [\[38\]](#) focuses on [KG](#) refinement methods such as entity resolution and ontology alignment, highlighting their importance in improving data quality after the initial mapping phase. As outlined in [Chapter 2](#), declarative mapping languages like [R2RML](#) [\[47\]](#) and its generalisation [RML](#) [\[13\]](#) play a key role in generating [RDF](#) data from heterogeneous sources, ensuring reproducibility and source traceability across different data formats. [RML-star](#) further extends this paradigm by enabling the creation of [RDF-star](#) triples that can include metadata or provenance statements directly as part of the graph structure [\[11, 18\]](#).

3.2 Mapping Engine Landscape

Since the publication of the [R2RML](#) recommendation in 2012 [\[47\]](#), several engines have been developed to operationalise the mapping of structured and semi-structured data into [RDF](#). Early [R2RML](#) processors such as Ontop [\[49\]](#) formed the basis for query-driven virtualisation, where [R2RML](#) mappings are translated into [Structured Query Language \(SQL\)](#) queries that generate [RDF](#) views on relational databases. These approaches avoided full materialisation of triples and leveraged existing database optimisers for query rewriting and execution.

The introduction of [RML](#) [\[13\]](#) broadened this landscape by supporting heteroge-

neous input formats beyond relational sources, prompting the emergence of a new generation of materialisation-focused engines. RMLMapper [12] became the reference implementation, designed for correctness and specification compliance rather than large-scale performance. Subsequent systems such as CARML [8] and RocketRML [42] aimed to simplify integration workflows by providing library-based interfaces and lightweight execution. In parallel, engines like Morph-KGC [2] extended the R2RML and RML paradigms by introducing partition-based materialisation and SQL pushdown optimisation, achieving scalability on large relational sources through reduced memory usage and parallel query execution.

RDF-star and RML-star support.

The recent extensions of RDF and RML toward RDF-star [18] and RML-star [11] enable the declarative generation of quoted triples that capture statement-level metadata such as provenance or confidence. Until recently, implementations of these specifications were largely experimental, with engines typically resorting to RDF reification or ad hoc templates to represent annotations.

A significant advance was introduced by Arenas-Guerrero et al., who extended the Morph-KGC engine to **Morph-KGC^{star}** [3], providing the first peer-reviewed implementation centered on RML-star materialization. Morph-KGC^{star} introduces constructs for quoted-triple generation in subject and object positions, supports nested quoted triples, and remains backward compatible with standard RML mappings. The paper validates the implementation with test cases derived from the N-Triples-star syntax tests together with two real-world use cases, and reports stronger performance than **SPARQL Anything** [9] on large-sized data while being slower on small-sized data in the tested scenarios [3]. Public project sources further present RDF-star/RML-star support as a built-in capability of Morph-KGC alongside broader RML functionality such as YARRRML, RML-FNML, and heterogeneous-source materialization [30, 28, 29].

These developments show that RDF-star materialization is no longer only a conceptual extension of RML, but an increasingly practical feature of modern mapping engines. At the same time, they reinforce the need for execution strategies that reuse quoted results efficiently when star-shaped mappings become prominent. Given these developments, the next sections provide a focused comparison of representative engines, highlighting their capabilities, limitations, and emerging support for RDF-star constructs.

3.3 Engines Without Native RML-star Support

3.3.1 RMLMapper

RMLMapper is an open-source **KG** materialization engine developed in the **RML.io toolchain**. It executes **RML mapping rules** to generate **RDF KGs** from heterogeneous data sources. **RML** itself was introduced as an extension of **R2RML** to support non-relational and semi-structured sources (e.g., **CSV**, **JSON**, **XML**), enabling integrated **RDF** mappings across formats. [13]

From an implementation perspective, the official RMLio GitHub documentation describes RMLMapper as a Java library that can also be used through a **Command-Line Interface (CLI)**. It supports multiple local input formats, including **CSV** (including **CSVW**), **JSON** (**JSONPath**), **XML** (**XPath**), and **spreadsheet files** (**Excel .xlsx** and **LibreOffice .ods**), and it also supports several remote sources, such as **relational databases** (**MySQL**, **PostgreSQL**, **Oracle**, **SQLServer**), **SPARQL endpoints**, **Web APIs** via the W3C Web of Things approach, and **HTTP-accessible files**. [20]

RMLMapper provides common **KG** construction features expected from an **RML** processor: join conditions, support for functions (via an **Function Ontology (FnO)**-style function description and configurable function loading), and several **RDF** output serializations including N-Quads (default), Turtle, TriG, TriX, JSON-LD, HDT, and Jelly [20]. It also supports metadata generation: the RMLMapper documentation explicitly links this capability to prior scientific work on automated provenance/metadata generation in the **RML** toolchain using standard vocabularies such as **PROV-O**. [12]

Finally, RMLMapper has been used in the community as a “reference” implementation for **RML**: a conformance study tested the RMLMapper against a large set of **RML** test cases and reported broad support across common formats (**CSV**, **XML**, **JSON**) and partial support for relational databases. [19]

Limitations

A central limitation is that RMLMapper **loads all data in memory**, and the maintainers explicitly warn to be careful with big datasets. This design choice can limit scalability when input data grows large, especially compared to stream-oriented or highly optimized engines. [20]

The official documentation notes that RMLMapper’s **XML** parsing approach was selected to support **full XPath**, but this can lead to **large memory consumption**

(reported up to $\tilde{10}\times$ the original XML file size). This is important when evaluating efficiency on XML-heavy pipelines. [20]

For newer “modular RML” work under the W3C Knowledge Graph Construction (KGC) Community Group (often referred to as “RMLkgc”), RMLMapper has **some support** and even provides a CLI option to **convert older R2RML/RML mappings to the newer version** [20]. However, the KGCW 2025 challenge report for RMLMapper states that they participated in **RML-Core, RML-IO, and RML-LV**, reporting approximate coverage levels of **90% (Core)** and **65% (IO)**, while **Logical Views (RML-LV)** required an added preprocessing step (RML-view-to-CSV) to reach higher coverage. The same report explicitly lists **RML-Star** as a distinct module in the modular ecosystem, but RMLMapper’s participation/results were not for RML-Star, and the authors note they cannot claim full coverage of the modular specification. [10]

3.3.2 CARML

CARML is an open-source Java mapping engine that materializes RDF from structured (semi-structured) sources according to an RML mapping, i.e., it executes RML rules to generate RDF. The project positions itself explicitly as an RML implementation (RML as a superset of R2RML) and is distributed primarily as a Java library, with a separate CLI “runnable jar” available for command-line execution. [8]

From an architectural perspective, CARML is built around RDF4J data structures: its default outputs are either (i) an **RDF4J Model**, or (ii) a **stream/sequence of RDF4J Statements**. In addition, the project provides utilities to convert this RDF4J-centric output into Apache Jena equivalents (e.g., mapping statements into Jena quads and collecting them into a Jena Dataset), which makes it easier to integrate CARML into Jena-based toolchains without rewriting the mapping process itself. [8]

A key design goal of CARML is streaming-friendly materialization. In earlier descriptions of the system within streaming/Big-Data-focused academic work, CARML is characterized as an RML engine that streams the generation process to address growing data volumes, aiming to mitigate “out of memory” issues typical of fully in-memory materialization. [15]

In current project documentation, CARML implements this idea using Reactive Streams via Project Reactor, exposing a `Flux<Statement>` result when executing mappings in streaming mode — so downstream processing can be chained using Reactor operators (and, depending on the rest of the pipeline, can be non-blocking). [8] CARML also provides a few practical extensions beyond “plain” RML through its

own `carml`: vocabulary.

- **Named streams (`carml:Stream`):** Instead of only reading logical sources from files, CARML can work with input streams (e.g., continuously arriving data) by declaring a stream source in the mapping and binding the actual stream at runtime. [8]
- **XML namespaces (`carml:declaresNamespace`):** For XML inputs, CARML allows declaring namespaces so that XPath expressions can be evaluated correctly. [8]
- **Functions (FnO-based):** CARML supports calling functions during mapping, following an approach aligned with the FnO as described in its documentation. [8, 4]

Limitations

- First, **no explicit RML-star/RDF-star support is documented** for CARML in its public documentation.¹ [8]
- Second, CARML introduces **engine-specific extensions** using the `carml`: vocabulary (e.g., `carml:Stream` and `carml:declaresNamespace`). While useful in practice, such extensions can reduce **portability** because other RML engines may not support these CARML-specific terms. [8, 4]
- Third, in the literature CARML is commonly described with support focused on common structured/semi-structured formats such as CSV, JSON, and XML; using other kinds of sources often requires additional integration or pre-processing. [8, 4]
- Fourth, CARML's **streaming/reactive execution** can help reduce memory pressure during materialization, but this does not automatically provide **distributed execution**. [15, 8]
- Finally, CARML supports **function-based transformations** (FnO-style), but documentation also indicates practical gaps between runtime capabilities and what is fully expressible/standardized in the function description model. [8]

¹CARML GitHub repository

3.3.3 RocketRML

RocketRML is an **RML** mapping engine implemented in JavaScript/Node.js that materializes **RDF** from heterogeneous data sources. It was introduced to enable practical **RDF** generation from **large, real-world tourism datasets** (notably **XML/JSON**), with a focus on improving throughput in scenarios where earlier Java-based mappers were not sufficiently efficient [42]. In the original description, RocketRML’s workflow builds an **intermediate JSON-LD representation** during mapping and then serializes it into **RDF**, which simplifies processing and transformation steps [42].

According to the authors’ GitHub documentation, RocketRML supports common logical source formats such as **JSON**, **XML**, and **CSV**, and relies on established selector mechanisms/libraries (e.g., JSONPath for **JSON**, XPath for **XML**, and **CSV** parsing libraries) [41]. The documentation also mentions configuration choices intended to improve compatibility and performance for **XML** processing (e.g., selecting alternative XPath engines) [41]. Additionally, the original paper highlights that RocketRML was adapted to handle **nested input structures** by recursively iterating over nested objects to produce the corresponding **RDF** structures [42].

Limitations

- **Partial coverage of the broader **RML/R2RML** feature space.** The public documentation indicates that some mapping constructs are not supported (e.g., `rr:GraphMap` and **SQL/R2RML** view-related constructs such as `rr:R2RMLView` and `rr:BaseTableOrView`), which can restrict usage in scenarios requiring named graphs or relational-source features [41].
- **Named graphs remain a known gap.** The original publication reported no support for named graphs/quads, and the repository issue tracker contains requests to add named graph support, suggesting this capability is still not fully available in practice [42, 41].
- **Joins can be challenging depending on the mapping pattern.** Early descriptions emphasize use-case-driven design choices and discuss limitations around features such as joins in initial versions [42]. Although the current documentation lists referencing object maps and join-related constructs as supported, user reports in the issue tracker indicate that join-related edge cases can still occur [41, 41].

- **Functions depend on JavaScript implementations and may have term-typing constraints.** RocketRML supports function-based extensions, but functions are implemented in JavaScript, and issue discussions show practical constraints (e.g., when a function result should be treated as an `IRI` rather than a literal/string) [42, 41].
- **Release engineering and maintenance signals.** The project does not provide versioned releases on GitHub, which can make reproducible evaluations and version pinning harder in comparative studies [41]. Related downstream work also suggests limited maintenance activity beyond bug fixes [41].
- **No documented `RML-star`/`RDF-star` generation support.** Since this thesis targets efficient `KG` generation from `RML-star` specifications, it is important that RocketRML’s public documentation focuses on classic `RML`/`RDF` materialization and does not describe native constructs for producing `RDF-star` quoted triples, as introduced by `RML-star`/`RDF-star` specifications [41].

3.3.4 Ontop

Ontop is an open-source `Virtual Knowledge Graph (VKG)` and `Ontology-Based Data Access (OBDA)` system. It allows users to query relational databases through a virtual `RDF` graph: the data stay in the database, and Ontop answers `SPARQL` queries by translating (rewriting and unfolding) them into `SQL` queries that are executed by the underlying `Database Management System (DBMS)` [6, 49]. This “virtual” design avoids pre-generating and storing a full `RDF` dump in many use cases, because `RDF` triples are produced only as needed to answer a query [6].

Ontop can also *materialize* an `RDF` graph, i.e., export the triples that can be generated by the mappings into `RDF` files (e.g., Turtle or N-Triples) using its `CLI` tooling [32, 33]. However, Ontop is primarily presented and optimized as a system for virtual query answering rather than as a bulk `RDF` generation pipeline [6, 49].

Mapping language focus (`R2RML`, and relation to `RML`).

Ontop relies on `R2RML`, the W3C recommendation for describing customized mappings from relational databases to `RDF` [47, 36, 33]. In contrast, `RML` was introduced as an extension of `R2RML` to support heterogeneous, non-relational sources (e.g., `CSV`/`JSON`/`XML`) in a unified mapping approach [13]. Therefore, Ontop is best understood as a mapping engine specialized for the *relational*/`R2RML` setting, not as a general-purpose `RML` processor.

Why Ontop is often considered efficient for SPARQL querying.

Ontop’s performance focus is on pushing work to the database engine. It translates **SPARQL** query operators (e.g., joins, selections, aggregations) into **SQL** so that the **DBMS** can execute them using its query optimizer and indexing [6, 49]. Ontop can also exploit *lightweight ontologies*, especially the **Web Ontology Language (OWL)** 2 QL profile, to support ontology-mediated access while keeping query answering tractable via query rewriting [31, 6, 33].

Status of RDF-star / SPARQL-star support (latest publicly documented state).

As of Ontop’s stable documentation around version 5.4.0, Ontop describes itself as using **RDF** 1.1 as its graph data model and supporting the majority of **SPARQL** 1.1 features; **RDF-star**/SPARQL-star is not listed as a supported standard feature [33, 34]. **RDF-star** support has been investigated as research/prototype work: Sundqvist’s MSc thesis reports a proof-of-concept extension of Ontop toward **RDF-star**/SPARQL-star and discusses the required changes in parsing, internal representation, and unfolding [43]. In addition, an Ontop maintainer publicly stated that the implementation was partial and not merged, and referenced an experimental **rdfstar** branch in the Ontop GitHub repository [1, 35].

Limitations

- **Relational and R2RML-centered (not a general RML engine).** Ontop’s core scope is relational databases mapped with **R2RML** [36, 33]. Since **RML** targets heterogeneous sources beyond relational data by extending **R2RML** [13], Ontop is not designed as an executor for general **RML** mappings over non-relational inputs.
- **No official RDF-star / SPARQL-star support in the stable toolchain.** Ontop’s official documentation emphasizes **RDF** 1.1 / **SPARQL** 1.1 compliance and does not list **RDF-star** as a supported feature in stable releases [33, 34]. Public project communication and the referenced **rdfstar** branch indicate experimental, unmerged prototype work rather than production support [1, 35]. This limits Ontop for use cases that require statement-level metadata modeled directly as quoted triples (e.g., provenance/temporal annotations in **RDF-star**) [43].
- **Materialization exists, but Ontop’s main design goal is virtual query**

answering. Ontop provides `CLI` support to materialize triples into `RDF` files [32, 33]. Still, the system’s main research and engineering focus is `SPARQL` to `SQL` query answering over a virtual graph, i.e., avoiding full precomputed dumps when possible [6, 49].

- **Performance depends on mappings, ontology profile, and DBMS behavior.** Ontop’s rewritten `SQL` can become complex when mappings are complex and when ontology-mediated querying is used [6, 49]. In practice, good performance often relies on careful mapping design and proper database-side optimization [6].

3.3.5 RMLStreamer

RMLStreamer is an open-source `RML` mapping engine that focuses on generating `RDF` from large files and, especially, from *continuous data streams* (e.g., sensor-like streams). It is implemented on top of Apache Flink: mappings are executed as stream-processing jobs, so `RDF` is produced incrementally while data keeps arriving, instead of requiring a full input dataset to be available upfront [37, 39]. The project documentation explicitly positions RMLStreamer as an `RML` processor for “big input files and continuous data streams” [39].

Streaming-oriented execution model.

In their ISWC 2022 paper, Oo et al. present RMLStreamer-SISO as a scalable engine for generating `RDF` streams from heterogeneous input streams, targeting low latency and high throughput. The approach relies on a *window-based* processing strategy so that the engine can continuously transform incoming events into `RDF` while still handling joins/associations across streaming inputs [37]. In practice, RMLStreamer can consume data via streaming logical sources (e.g., TCP sockets, Kafka) and emit `RDF` continuously [39].

RML features and extensions used for streaming.

To describe streaming inputs in mappings, the RMLStreamer documentation introduces an extension vocabulary (namespace `rmls:`) for *streaming logical sources*, including terms such as `rmls:TCPSocketStream` and `rmls:KafkaStream` together with connection details (host/port) [39]. This complements standard `RML` constructs (logical source + reference formulation such as `JSONPath`), but is tailored to the streaming setting [39].

Recent public project status (release-level).

According to the project releases page, the latest tagged release is v2.5.0 (June 2, 2023). The release notes highlight added support for relational databases via `Java Database Connectivity (JDBC)` and a `parallelism` parameter that sets the maximum number of parallel operations [16]. These changes are relevant for performance engineering, because they make parallel execution more configurable and widen supported sources [16].

Limitations

Higher operational overhead than batch-oriented engines.

Because RMLStreamer runs as a Flink-based streaming job, it typically requires additional setup (runtime configuration, connectors, stream endpoints such as sockets/Kafka, logging/checkpointing decisions). This overhead can be disproportionate for small, static datasets where a lightweight batch materializer would be simpler [39, 37].

Duplicate handling and output semantics can require extra care.

Users have discussed output duplicate handling in the project issue tracker (e.g., questions about removing duplicates from output), indicating that deduplication may require explicit handling outside the basic mapping definition [40].

No documented native RDF-star (quoted triples) output pipeline.

`RML-star` targets `RDF-star` generation [11, 25], but RMLStreamer’s documentation and release notes do not state support for `RDF-star`/`RML-star`. Hence, `RDF-star`-based pipelines should treat such support as unconfirmed and expect extra engineering or a different engine unless a citable implementation statement is provided [39, 16].

3.4 Engines Supporting RML-star / RDF-star

3.4.1 SDM-RDFizer (RML-star support)

SDM-RDFizer is an open-source `RML` materialization engine for constructing `RDF` `KGs` from heterogeneous data sources [23]. In this subsection, we focus only on its `RML-star` / `RDF-star` standing and on the specific baseline characteristics

that motivate the solution proposed in this thesis. The general execution architecture, planning phases, and internal data structures of SDM-RDFizer were already introduced in Chapter 2.

Current RML-star standing.

In the KGCW 2024 results report, SDM-RDFizer is described as fully supporting the RML-star test category. The report explains that this support is achieved by extending the engine with an operator for generating inner, i.e., quoted, triples during materialization [24]. This establishes SDM-RDFizer as a practical baseline for producing RDF-star graphs from RML-star mappings.

Author-acknowledged efficiency gap.

The same report also identifies an important open efficiency issue in the current approach. More specifically, it proposes a new intermediate-results data structure for RML-star processing in order to avoid repeated generation of inner triples [24]. This is highly relevant for the present thesis, because it makes redundant reconstruction of quoted triples an explicit optimization target already recognized by the SDM-RDFizer authors themselves.

Observed baseline limitations.

Beyond this author-acknowledged efficiency gap, our implementation work on the baseline engine revealed some concrete limitations that are relevant for RDF-star heavy mappings.

- **L1:** The baseline runtime may repeatedly reconstruct quoted intermediate results instead of reusing them systematically across later triples maps. This is especially visible in workloads where the same quoted statement is referenced multiple times, either within the same logical source or across joins. As a consequence, the practical cost of RDF-star materialization is influenced not only by the number of output triples, but also by how often quoted intermediate results are rebuilt and retained.
- **L2:** Second limitation: The naive implementation showed a concrete correctness limitation on a nested cross-source RML-star mapping in which the final outer rule connected two quoted results: the outer subject was itself quoted, the outer object was also quoted, and both sides were derived through supporting quoted mappings. In this setting, the engine did not terminate with an exception, but it omitted the final outer alignment layer entirely. This is an important limitation because it shows that, for certain recursively dependent

RDF-star mapping patterns, the issue is not only repeated computation but also the correct composition of quoted intermediate results into the final outer statements.

- **L3:** Third limitation: Another execution-stage limitation of the naive SDM-RDFizer implementation appears when the same quoted statement is reused by several later cross-source rules. In the observed reproduction case, the first reuse branch is materialized correctly, whereas a later branch over the same quoted producer is omitted. This behavior is tied to the way the naive runtime reuses its existing duplicate-detection structures. In particular, quoted producer materialization for join support is not isolated from the global duplicate table, the Predicate Tuple Table (**PTT**), which stores predicate-keyed subject-object combinations for duplicate elimination. As a consequence, when an earlier branch materializes the shared quoted producer, the corresponding combinations may already be registered in the **PTT**. A later branch that attempts to rebuild a quoted join index over the same producer may therefore find no remaining producer tuples available for its own join-support construction, even though the mapping itself is well formed. The result is not duplicate output, but missing valid output triples. This limitation is important for the thesis because **RDF-star** execution requires not only constructing quoted intermediate results, but also reusing them reliably across multiple dependent rules while preserving duplicate elimination.

Why these limitations matter.

Taken together, these observations show that the challenge is not simply to “add **RDF-star** support” to an **RML** engine. Once quoted triples become intermediate results that are reused, joined, and embedded again, the engine must manage them as first-class runtime objects. Without a dedicated reuse strategy and tighter control over their lifetime, repeated reconstruction and fragile composition of quoted results can become a practical bottleneck and, in some cases, a correctness issue.

Thesis contribution (final solution).

Building on the newest SDM-RDFizer baseline distributed through the project repository and releases [22], this thesis adopts the **Unified cache design** as its final solution for more efficient **RDF-star** materialization even for complex **RML-star** mappings. Rather than replacing SDM-RDFizer’s overall execution skeleton, the proposed solution extends it with a unified reuse strategy for quoted results, planner-guided execution decisions, and tighter control over intermediate cache lifetimes while

preserving duplicate suppression. The goal is to reduce redundant quoted-triple reconstruction while preserving correctness and **RML-star** compliance [24]. The detailed design and evaluation of this solution are presented in the following chapters. The following chapter presents the detailed design of the proposed solution and explains how it addresses the identified limitations **L1**, **L2**, and the remaining observed limitations while building on top of the existing implementation. The evaluation chapter then compares the naive and unified implementations concretely, using both output-level and performance-oriented results to show how these limitations are resolved in practice.

3.4.2 Morph-KGC

Morph-KGC is an open-source **KGC** engine that materializes **RDF KGs** from heterogeneous data sources using declarative mappings in **R2RML** and **RML** [2, 30, 28]. It is implemented in Python and can be used either as a command-line tool driven by an INI configuration file or as a Python library [30, 28]. Public project sources present integrations with RDFLib, Oxigraph, and other output targets [30, 28, 29].

A key idea behind Morph-KGC is *mapping partitions*. The engine analyzes the mapping rules and groups them into partitions that can be processed separately, with the goal of reducing peak memory usage and improving execution time for large materialization tasks [2]. In the Morph-KGC evaluation, Arenas-Guerrero et al. report that this partition-based strategy can significantly improve materialization performance and reduce memory usage compared with other approaches on standard benchmarks [2].

Support for **RDF-star** / **RML-star**.

Beyond plain **RDF** generation, Morph-KGC has explicit support for generating **RDF-star** graphs from **RML-star** mappings [3, 30, 28, 29]. The Morph-KGCstar work presents an extension of Morph-KGC for processing **RML-star** and generating **RDF-star** datasets, and validates the implementation using test cases derived from the N-Triples-star syntax tests together with two real-world use cases from the biomedical and open science domains [3].

Recent implementation status.

From a software-distribution perspective, **RDF-star** generation is presented as a built-in feature in the main Morph-KGC GitHub repository, documentation, and PyPI package description [30, 28, 29]. As of April 22, 2026, the latest verified PyPI release is version 2.10.0, released on January 20, 2026 [28]. Public project materials and documentation also present support for YARRRML alongside **RML-star** [30, 28, 29].

Limitations

Performance depends on workload shape.

The reported performance of Morph-KGCstar depends on the workload and comparison setting. In the **RDF-star** evaluation, the authors report that Morph-KGCstar outperforms SPARQL-Anything on large-sized data, but is slower on small-sized data in the tested scenarios [3]. This suggests that its practical performance should be interpreted with respect to workload shape rather than assumed to be uniformly superior across all cases.

Python-based execution trade-offs.

Morph-KGC is implemented in Python and explicitly built on top of pandas [30, 28]. This enables expressive and practical data processing over heterogeneous inputs, but also means that runtime behavior is shaped by Python and pandas execution characteristics, including dataframe materialization costs, memory pressure, and environment-dependent overheads. This should not be read as a claim of poor scalability: on the contrary, the Morph-KGC evaluation reports strong performance and favorable scaling on several benchmark settings [2]. Rather, it indicates that deployment decisions may still depend on dataset size, execution environment, and workload structure.

Observed implementation-level limitation in our experiments.

In our experiments with Morph-KGC 2.10.0, we observed a reproducible failure on a small **RML-star** case in which an outer mapping rule simultaneously used a quoted-join subject and a quoted-join object. A neighboring control case with only one quoted-join side completed successfully, whereas the double quoted-join variant failed in Morph-KGC with a pandas merge/join error indicating overlapping intermediate column names (`ValueError: columns overlap but no suffix specified`). This should be interpreted as an implementation-level limitation observed in version 2.10.0, rather than as a claim about **RML-star** expressiveness in general.

3.5 Summary

This chapter reviewed the related work most relevant to the thesis, covering the broader mapping-engine landscape as well as representative systems with and without native **RML-star** support. The comparison shows that existing engines differ not only in supported source types and execution models, but also in how far they currently support **RDF-star** materialization in practice.

A central observation is that native support for `RML-star` does not by itself guarantee robust execution on complex star-shaped mappings. In SDM-RDFizer, both prior work and our own implementation study indicate that quoted intermediate results remain a key challenge, affecting both efficiency and, in some mapping patterns, correctness. In Morph-KGC, our experiments additionally revealed an implementation-level failure on a double quoted-join pattern. These observations motivate the need for a more systematic execution strategy for quoted-result reuse and composition.

The next chapter presents the proposed unified solution, which builds on the existing SDM-RDFizer implementation and is designed to address the identified limitations in a principled way.

Chapter 4

Approach and Proposed Solution

This chapter presents the design of the unified execution strategy used in SDM-RDFizer-Plus. The motivation, problem statement, and motivating example were introduced in Chapter [1](#); this chapter therefore focuses only on the design decisions needed to address them.

The proposed strategy keeps the original SDM-RDFizer organization: [TMP](#) followed by [TME](#). The change is that quoted [RML-star](#) results are no longer treated as temporary by-products of ordinary triple generation. Instead, they are planned, retained, reused, indexed, and released as explicit intermediate payloads.

4.1 Design Goals

The unified strategy is guided by six design goals.

1. **Preserve mapping semantics.** Reuse and caching must not change the [RDF-star](#) graph defined by the mappings. The optimized execution must preserve valid asserted and quoted output.
2. **Make quoted-result dependencies explicit.** The planner must identify which triples maps produce quoted results and which later triples maps or predicate-object maps consume them.
3. **Avoid unnecessary reconstruction.** If several later rules require the same quoted statement, the engine should reuse the retained payload instead of rematerializing it independently for each branch.

4. **Support same-source and cross-source reuse.** Reuse must work both when a consumer refers to a quoted result from the same source row and when the result is reached through a join with another source.
5. **Preserve duplicate suppression.** Duplicate final statements should still be suppressed, but duplicate suppression must not remove reusable quoted payloads that are needed by later mappings.
6. **Control intermediate-result lifetime.** Quoted payloads should be retained only while dependent consumers may still need them. Cleanup should therefore be guided by dependency information rather than by local execution order alone.

4.2 Dependency Analysis

The first design step is dependency analysis over triples maps and predicate-object maps. A triples map is treated as a quoted producer when its generated statement may be referenced by another mapping rule. A triples map or predicate-object map is treated as a consumer when it refers to such a producer as a quoted subject, quoted object, or quoted join target.

This analysis is needed at two levels. At the triples-map level, it reveals which maps must be executed before others and which quoted payloads may be needed beyond the producer’s own execution. At the predicate-object-map level, it avoids overly coarse decisions: different predicate-object maps inside the same triples map may depend on different quoted producers or use different access modes.

The dependency analysis records three kinds of information:

- the producer–consumer relationships between triples maps;
- the access mode used by each consumer, such as same-row reuse, quoted-subject reuse, quoted-object reuse, or join-based reuse;
- the remaining consumers that determine when a retained quoted payload can be safely released.

4.3 Statement Equivalence

Some mappings construct the same statement-level payload through different local structures. For example, a non-asserted support triples map may produce the same quoted statement that is also represented by an owner-side predicate-object-map

slice. If these equivalent structures are not recognized, the engine may duplicate work or fail to reuse a payload through the correct access path.

The unified strategy therefore includes statement-equivalence detection. It assigns canonical identities to equivalent statement slices so that reusable payloads can be retained once and accessed through the relevant owner-side or support-map views. This is especially important for mappings that combine same-row reuse with join-based reuse.

4.4 Unified Reuse Layer

The execution design uses one coordinated reuse layer rather than separate ad-hoc handling for each quoted access pattern. This layer combines row-keyed payload stores with join-keyed lookup views.

Row-keyed stores retain the quoted payload produced for a specific source row or canonical statement slice. These stores support direct reuse when a later mapping needs the quoted result from the same execution context.

Join-keyed lookup views record which retained producer rows are reachable through a join value. These views allow cross-source consumers to find the quoted payloads needed for their own output without reconstructing the producer independently.

The reuse layer separates two responsibilities that were previously entangled: retaining a quoted payload for later consumers, and deciding whether an asserted triple should be emitted to the final **KG**. The **PTT** can still suppress duplicate final output, but it should not be the mechanism that decides whether a reusable quoted payload exists.

4.5 Cache Lifetime

Because quoted payloads can become large, retaining all of them indefinitely would not be practical. The unified strategy therefore uses dependency-aware cache lifetime management. During planning, the engine determines which later consumers still need each quoted producer or equivalence group. During execution, cache entries can be released once those consumers have completed.

This design is intentionally conservative with respect to correctness. A payload should not be flushed while any dependent mapping may still need it. However, the strategy still avoids unconditional retention by linking cleanup to the dependency structure discovered during planning.

4.6 Expected Benefits

The design is expected to improve RML-star materialization in three ways. First, it reduces repeated reconstruction of quoted statements by retaining and reusing intermediate payloads. Second, it improves correctness on layered and cross-source mappings by ensuring that quoted results remain available to all dependent consumers. Third, it gives the engine a principled basis for cache cleanup, which is necessary to control memory pressure.

The evaluation chapter tests these expectations against the naive SDM-RDFizer implementation and Morph-KGC. In particular, it checks whether the unified strategy preserves correctness on the layered benchmark family and how much memory is required to retain the quoted payloads that make this correctness possible.

Chapter 5

Implementation

5.1 Implementation of the Unified Execution Strategy

The unified implementation follows the same high-level two-phase organization as SDM-RDFizer: Triples Map Planning (TMP) followed by Triples Map Execution (TME). The goal is not to replace the original materialization engine, but to extend its existing planning and execution phases with star-aware mechanisms for quoted intermediate results. Therefore, the implementation is presented phase by phase. For each phase, we first describe the structures and behavior preserved from SDM-RDFizer and then explain the unified contribution added in this thesis.

The goal of this thesis is not to replace the naive SDM-RDFizer implementation because it is generally inefficient. On the contrary, the baseline execution strategy is already fast and memory-efficient on several simpler RML-star mappings. However, the analysis showed that this efficiency is partly achieved with execution logic that is not sufficiently robust for more complex quoted-result dependency patterns. In such cases, the engine may omit valid output triples or handle reusable quoted results in an order-sensitive way.

Therefore, the proposed unified implementation should be understood as a star-aware refactoring and extension of the existing engine. It preserves the efficient parts of the original TMP–TME execution model, while adding explicit dependency analysis, managed quoted-result reuse, and dependency-aware cache cleanup. The objective is to retain the practical efficiency of the naive implementation where it already works well, while improving completeness and robustness for complex RML-star mappings.

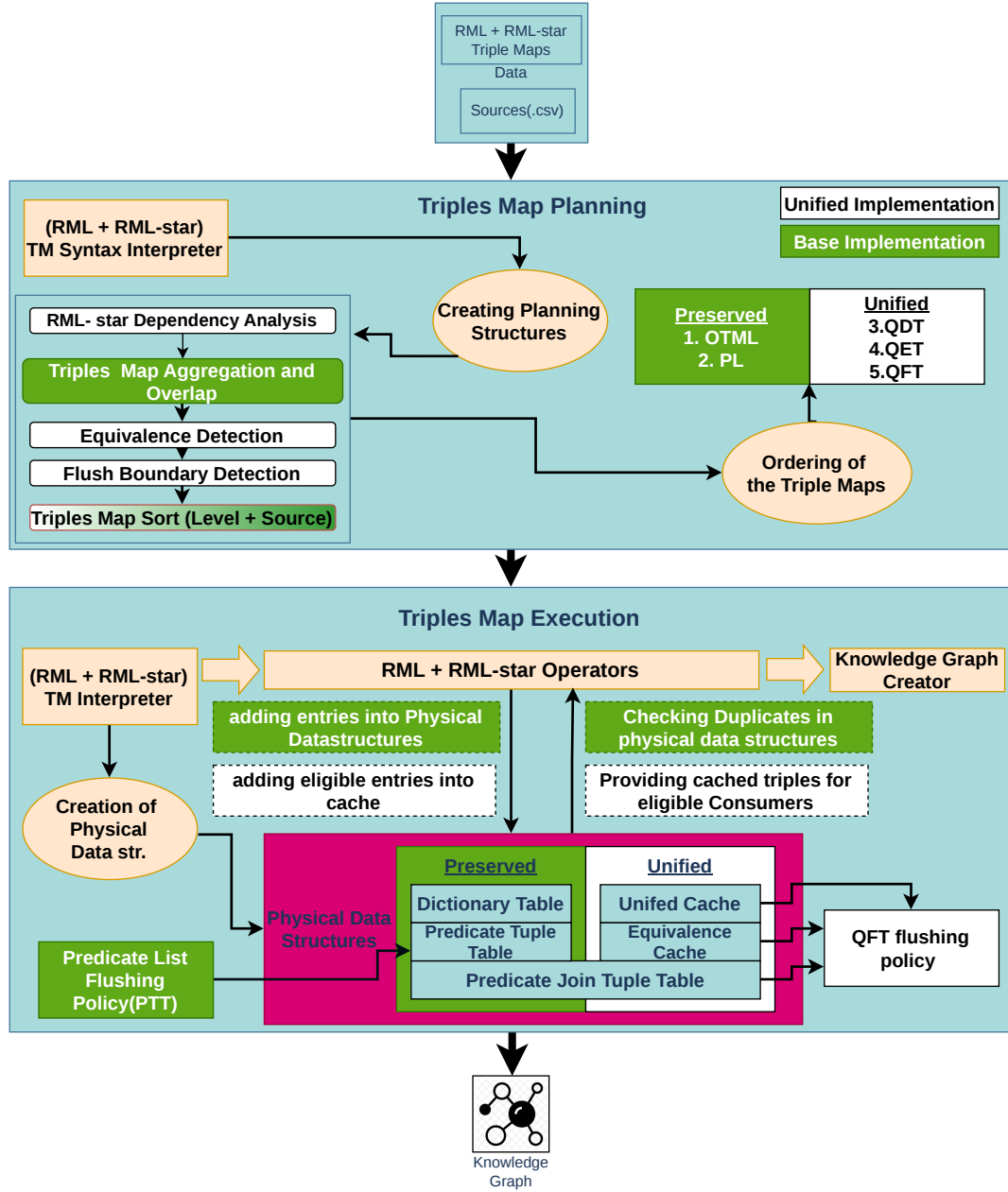


Figure 5.1: Architecture of the unified RML-star implementation. The system extends the base RDFizer-style materialization workflow with planning-context construction, TM-level dependency analysis, quoted reuse, unified cache flushing.

5.2 Planning Phase: From Source Organization to Star-Aware Planning

The planning phase is the point at which the unified implementation first makes the RML-star-specific execution problem explicit. In ordinary materialization, a triples map is mainly planned as a rule that contributes statements to the final knowledge graph. In RML-star materialization, however, a triples map may also act as a *quoted-result producer*: it generates a statement that is later used as a quoted subject or quoted object by another mapping rule. The triples map or predicate-object map that uses this quoted result is called a *quoted-result consumer*.

A common modeling convention in RML-star mappings is to introduce small non-asserted support triples maps for statements that may need to be quoted later. This is useful when only one statement produced by a larger triples map is needed as a quoted result. For example, an asserted triples map may contain several predicate-object maps, but a later rule may need to quote only the statement produced by one of them. In such a case, a separate non-asserted triples map can describe precisely the quoteable statement fragment, without treating all predicate-object maps of the larger triples map as one quoted result.

This convention creates an important planning issue: a non-asserted support triples map may describe the same statement-level payload as one predicate-object map inside a larger asserted triples map. In the setting considered in this thesis, the planner records this situation as an **equivalence** relation between the support triples map and the owning triples-map/predicate-object-map pair. During execution, this allows the engine to reuse an already materialized payload instead of recomputing the same quoted statement through the support map, while duplicate suppression is still preserved.

The producer–consumer relation can also occur at different depths. A direct dependency exists when a consumer immediately refers to a quoted producer through `rml:quotedTriplesMap`. A transitive dependency exists when a quoted result is consumed by another support mapping, which is then consumed again by a later mapping. These transitive chains are important for lifetime management, because a producer may still be needed even after its direct consumer has been processed.

5.2.1 Preserved Planning Structures

The unified implementation preserves the original planning responsibilities of SDM-RDFizer. In the TMP phase, the engine still parses the mapping file into triples map objects and organizes these triples maps before execution. This preserved planning

layer remains responsible for source-based organization, predicate-oriented information, and the basic execution order used by the TME phase.

First, triples maps are still grouped according to their logical sources. This preserves the original OTML-style organization of the engine, where mappings over the same source can be processed together during execution. As in the base SDM-RDFizer formulation, OTML should be understood here as a planning data structure that captures source-oriented organization rather than as a separate low-level runtime table. In the unified implementation, this source-oriented organization remains the basis for iterating over CSV files and applying the relevant triples maps to their rows.

Second, predicate-oriented planning information is still preserved. In SDM-RDFizer, predicate information is used to support duplicate suppression and to decide when entries in the predicate tuple table can be released. The unified implementation keeps this mechanism because duplicate suppression is still required for both ordinary RDF statements and RDF-star statements.

Third, the unified planner preserves the original OTML-style source ordering, but refines it with dependency levels. Conceptually, triples maps are first organized into dependency levels and then ordered according to OTML within each level. Thus, level planning does not replace source-oriented ordering. Rather, it localizes OTML inside each dependency level, so that execution becomes level-first while preserving source-based grouping among the triples maps that belong to the same level.

Thus, the preserved TMP phase provides the same foundation as in SDM-RDFizer: parsed triples maps, source-based grouping, predicate-oriented cleanup information, and execution order. The unified contribution extends this foundation with additional planning data for quoted dependencies, equivalence, and cache lifetime.

5.2.2 Unified Contribution in TMP

The unified contribution in the TMP phase is the collection of additional planning information for quoted-result execution. This information answers three questions: which quoted results depend on which producers, which statement-level payloads are equivalent, and how long quoted intermediate results must remain available.

The following tables are conceptual planning structures used to explain the information collected during TMP. They are not separate physical tables exposed by the implementation. Instead, the implementation stores this information together inside the unified planning context. The conceptual separation is used to make the planning responsibilities clear: QDT describes dependency, QET describes equivalence, and QFT describes lifetime and flushing.

Table 5.1: Conceptual planning structures introduced by the unified TMP extension.

Structure	Main question
QDT	Who depends on whom?
QET	Which results are equivalent?
QFT	When can results be released?

Quoted Dependency Table (QDT).

The QDT is defined from the viewpoint of the quoted-result producer. Each QDT entry represents one triples map that can generate a quoted intermediate result, and records the planning information needed to understand how that producer is used later in the mapping. In this way, the producer remains the reference point of the entry, while consumer-side information is recorded only as information about the later use of that producer.

At the triples-map level, the QDT identifies the producer triples map itself, its type (asserted or non-asserted), and the triples maps that directly consume its quoted result. It also records the producer role, that is, whether the producer acts only as a quoted support producer, only as a consumer of other quoted producers, or as both. At the predicate-object-map level, the QDT records the dependency location inside each consumer, that is, whether the producer is referenced through the consumer’s subject map or through one of its predicate-object maps, and whether that use appears in quoted subject position, quoted object position, or through a join.

The QDT further records direct consumer relationships, the source relation between producer and consumer, and the join fields when row alignment is required. This is necessary because some quoted dependencies are resolved within the same source row, while others are resolved through same-source or cross-source join paths. For producers that are consumed more than once, the QDT also captures whether reuse occurs only through same-row access, only through join-based access, or through a mixed access pattern.

Finally, the QDT records the dependency level of each producer in the quoted dependency graph. A level describes the producer’s position in the quoted producer–consumer landscape: producers without quoted dependencies appear at the lowest level, while mappings that depend on quoted results from lower levels appear at higher levels. These levels are planning metadata; they do not mean that every non-asserted triples map is executed independently as a top-level output-producing rule.

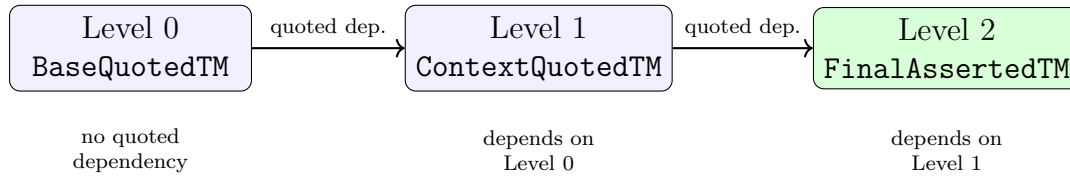


Figure 5.2: Example of quoted dependency levels in the planning phase. A triples map without quoted dependencies starts at level 0. A triples map that consumes a level 0 quoted result is placed at level 1, and a later consumer is placed at level 2.

Instead, the levels refine the preserved OTML-style source organization of the engine. Conceptually, triples maps are first grouped into dependency levels and then follow source-oriented ordering within each level. In this way, the QDT not only captures which producers depend on which consumers, but also provides the level-aware structure that later determines how triples maps are ordered for execution. .

Table 5.2: Concrete producer-centered QDT entry for a two-triples-map join example.

Information	Example value
Producer TM	RatedMovieTM
Producer type	asserted
Producer role	quoted_support
Direct consumer TM(s)	MovieAuditTM
Consumer-side dependency location	first predicate-object map (<code>po_index = 0</code>)
Dependency role	quoted object
Source relation	cross-source
Join fields	<code>child = [movieId], parent = [movieId]</code>
Access profile	join-based
Dependency level	0

To make the producer-centered viewpoint of the QDT more concrete, consider two triples maps. Let `RatedMovieTM` read a ratings CSV source and produce statements of the form « `:userX ex:rated :movieY` ». Let `MovieAuditTM` read a second CSV source and consume this quoted result through a join on `movieId`. From the producer viewpoint, the QDT entry for this case would be as shown in Table 5.2. The example shows the conceptually relevant fields of the QDT entry. Additional derived helper

metadata may also be maintained during planning, but is omitted here for readability.

```
<#RatedMovieTM> a rml:AssertedTriplesMap;
  rml:logicalSource [
    rml:source "data/ratings.csv";
    rml:referenceFormulation ql:CSV
  ];
  rml:subjectMap [
    rml:template "http://example.org/user/{
      ↪ userId}"
  ];
  rml:predicateObjectMap [
    rml:predicate ex:ratingValue;
    rml:objectMap [
      rml:reference "rating"
    ]
  ];
  rml:predicateObjectMap [
    rml:predicate ex:ratedMovie;
    rml:objectMap [
      rml:template "http://example.org/movie
      ↪ /{movieId}"
    ]
  ]
].
```

(a) RatedMovieTM

```
<#RatedMovieStatementSupportTM> a
  ↪ rml:NonAssertedTriplesMap;
  rml:logicalSource [
    rml:source "data/ratings.csv";
    rml:referenceFormulation ql:CSV
  ];
  rml:subjectMap [
    rml:template "http://example.org/user/{
      ↪ userId}"
  ];
  rml:predicateObjectMap [
    rml:predicate ex:ratedMovie;
    rml:objectMap [
      rml:template "http://example.org/movie
      ↪ /{movieId}"
    ]
  ]
].
```

(b) RatedMovieStatementSupportTM

Figure 5.3: Statement-level equivalence between the non-asserted `RatedMovieStatementSupportTM` and the `pomRatedMovie` slice of the asserted `RatedMovieTM`. Both describe the same quoteable `ex:ratedMedium` payload, allowing the planner to reuse the materialized statement.

Quoted Equivalence Table (QET).

The QET records the specific statement-level equivalence pattern considered in this thesis. In particular, it captures when a small non-asserted support triples map isolates the same quoteable payload that is already produced by one predicate-object map of a larger asserted triples map.

Each QET entry therefore links a support triples map to a canonical owner pair consisting of an owner triples map and an owner predicate-object-map index. This notion of equivalence was illustrated earlier in Figure 5.3. In that example, the non-asserted `RatedMovieStatementSupportTM` describes the same quoted payload as the `pomRatedMovie` slice of the asserted `RatedMovieTM`. The QET records this relation explicitly so that the execution phase can reuse the owner payload instead of rematerializing the same statement through the support triples map.

Table 5.3 should be read as one conceptual equivalence entry produced during planning. The planner derives an exact statement-shape fingerprint for each triples-map/predicate-object-map combination from the logical source context, subject-map shape, predicate-map shape, and object-map shape. It then checks whether a single-POM non-asserted support triples map has the same fingerprint as one predicate-object map inside another triples map. When such a match is found, the planner records an equivalence-group entry together with a canonical owner triples map and owner predicate-object-map index.

Table 5.3: Conceptual QET entry for the `RatedMovieTM` support-map equivalence example.

Field	Meaning	Stored value
Equivalence group ID	Identifier of the statement-level equivalence group	<code>RatedMovieTM::pom::0</code>
Equivalent support TM	Non-asserted support triples map that isolates the quoteable statement	<code>RatedMovieStatementSupportTM</code>
Canonical owner TM	Asserted triples map that owns the reused payload	<code>RatedMovieTM</code>
Canonical owner POM	Predicate-object-map slice of the owner TM that produces the same payload	<code>pomRatedMovie</code>
Canonical owner POM index	Planning index of the owner predicate-object map	0

Quoted Flush Table (QFT).

The QFT records the flush boundaries that determine how long quoted intermediate results stored in runtime caches must remain available during execution. It is derived from the dependency information collected in the QDT and, where applicable, from the equivalence information collected in the QET. Its purpose is to avoid two opposite problems: releasing cached results too early, which may cause missing output, and

Field	Meaning	Example value
Producer ID	Quoted producer whose cached result is being tracked	RatedMovieTM
Flush boundary	Latest safe release point as signed during planning	1

Table 5.4: Conceptual QFT entry for a producer-level flush boundary in the unified cache.

retaining them longer than needed, which may increase memory usage.

At the producer level, the QFT stores one flush boundary for each quoted producer triples map. This producer-level boundary is derived from the quoted producer-consumer structure identified during planning and represents the latest dependency stratum at which the producer may still be needed. Conceptually, it is computed from the producer’s own planning position together with the propagated flush boundaries of its quoted consumers. Thus, the producer-level flush boundary is propagated through downstream quoted consumer chains.

This should be understood as a conceptual planning structure rather than as a literal runtime counter mechanism. In particular, the system does not maintain an explicit numbered runtime level variable that is incremented during execution. Instead, the planner assigns dependency levels to triples maps in order to summarize their relative positions in the quoted dependency graph, and the QFT is derived from that planner-assigned structure.

In addition, the QFT records the first-use and last-use positions of a producer based on the planning levels of its direct consumers. These values summarize the span in which the producer is directly relevant, while the flush boundary gives the latest safe release point under the conservative dependency-based interpretation.

Let $\ell(t)$ denote the planner-assigned dependency level of a producer triples map t , and let $\text{Cons}(t)$ denote its direct quoted consumers. Then the producer-level flush boundary can be expressed recursively as

$$F_{\text{TM}}(t) = \max\left(\{\ell(t)\} \cup \{F_{\text{TM}}(c) \mid c \in \text{Cons}(t)\}\right).$$

This recursive definition is intended as a planning-time abstraction of the dependency-based lifetime computation. It captures the conceptual propagation of cache lifetime through the producer-consumer graph, rather than a literal step-by-step runtime update rule.

For an equivalence group g with members $\text{Members}(g)$, the group-level flush boundary is

$$F_{\text{EQ}}(g) = \max\{F_{\text{TM}}(m) \mid m \in \text{Members}(g)\}.$$

At the equivalence level, the QFT stores one flush boundary for each equivalence group recorded in the QET. This group-level boundary is derived from the producer-level flush boundaries of the participating members of the same equivalence group. Conceptually, it is the maximum producer-level flush boundary among the support mapping and its owner-side equivalents. This ensures that equivalence-aware cache entries are retained until the shared statement-level payload is no longer needed by any member of the group.

Field	Meaning	Example value
Canonical group ID	Owner-side identifier used to anchor the equivalence group	<code>RatedMovieTM::pom::0</code>
Flush boundary	Latest safe release point assigned during planning	1

Table 5.5: Conceptual QFT entry for the equivalence-group flush boundary with the canonical group ID `RatedMovieTM::pom::0` mentioned before in QET.

The QFT therefore stores these boundaries during planning, while the execution phase later interprets them against the remaining asserted execution frontier. When a producer has only direct asserted consumers, and all of those consumers have already completed, its cache entries may be released accordingly.

However, this direct completion view becomes more subtle in the presence of direct non-asserted support consumers. Such support mappings are not scheduled as top-level execution units in the main asserted execution order. Instead, they are materialized lazily when referenced by later asserted mappings. Consequently, execution cannot in general decide flush safety simply by asking whether such a non-asserted consumer has completed, because that support mapping does not appear as an explicit runtime milestone in the asserted execution frontier.

For this reason, the planner-derived flush boundary remains the governing rule for the general case in which direct non-asserted consumers are present. This conservative interpretation is especially necessary when such support mappings do not follow the equivalence pattern recognized by the QET, since execution cannot safely

infer from the asserted frontier alone whether the cached producer result may still be required indirectly.

The unified implementation refines this conservative rule for one specific safe case. If a direct non-asserted support mapping is recognized as a single-POM equivalence-style alias of a canonical asserted owner slice, then the non-asserted support mapping need not block flushing in the same way. Once that canonical asserted owner slice has already completed, the support mapping is treated as covered by the owner-side result. In effect, although the non-asserted support mapping is not itself a top-level runtime unit, its relevant quoted payload is already guaranteed to remain available through the completed asserted owner slice.

Thus, execution now distinguishes two cases for direct non-asserted consumers:

1. If the direct non-asserted support consumer is not recognized as an equivalence-backed alias, execution falls back to the conservative planner-derived QFT boundary.
2. If the direct non-asserted support consumer is recognized as an equivalence-backed single-POM alias whose canonical asserted owner slice has already completed, it no longer blocks early flushing of the producer cache.

This refinement does not replace the QFT. Rather, it changes how the QFT is interpreted at execution time. The QFT still supplies the conservative lifetime boundaries, but equivalence information from the QET allows execution to recognize a restricted class of non-asserted support consumers whose lifetime is already safely subsumed by a completed asserted owner slice.

In implementation terms, the QFT does not govern only an abstract producer lifetime, but also the release of concrete producer-scoped runtime structures. One important example is the Predicate Join Tuple Table (PJTT), referred to in the implementation as the join table. This table stores join-oriented intermediate lookup structures used during quoted and non-quoted referencing-object resolution. Once a producer becomes flush-eligible under the QFT interpretation, the corresponding PJTT entries owned by that producer may be released together with the other producer-scoped quoted caches. In this sense, the PJTT belongs to the QFT-controlled flush scope.

This PJTT cleanup should be distinguished from predicate-tracking cleanup, which is handled separately at triples-map completion through predicate-list release [23]. Predicate-list cleanup follows predicate-usage considerations, whereas PJTT release follows dependency-based cache lifetime and is therefore governed by the QFT.

Thus, the QFT distinguishes two related flush scopes: producer-level flush boundaries for the unified cache, including producer-owned PJTT entries, and group-level flush boundaries for the equivalence-aware caches. In this way, it turns the dependency structure from the QDT and the equivalence structure from the QET into explicit cache-lifetime decisions.

Planned Quoted Execution Order

After constructing the planning metadata, the unified implementation derives an execution order for asserted triples maps. In doing so, it preserves the basic execution style of the naive SDM-RDFizer engine: asserted triples maps define the explicit top-level runtime schedule, while non-asserted support mappings are not inserted into that schedule as ordinary execution units and remain available only through lazy support-side materialization when needed. This order combines two planning views: dependency levels from the quoted dependency analysis and the preserved source-oriented organization captured conceptually by OTML.

The first organizing principle is the quoted dependency level. Levels express the relative position of triples maps in the quoted producer-consumer graph, so that lower-level quoted producers appear before later consumers that depend on them. In this way, the planning phase introduces an execution layering only where quoted dependencies require it.

The second organizing principle is OTML source ordering. Within each dependency level, triples maps still follow the source-oriented grouping principle inherited from the base planning model. Thus, level planning does not replace source ordering. Rather, it refines it by making execution level-first while preserving source-based ordering among triples maps that coexist within the same level.

This also means that mappings without quoted dependency separation do not lose the source-ordered behavior of the original RML execution model. In ordinary RML-style cases, where no additional quoted levels are induced, execution continues to benefit from the same source-oriented grouping strategy as in the base engine. The level mechanism therefore adds structure only where quoted dependencies require it, while retaining the original source-ordered execution advantages elsewhere.

To make this planning result concrete, consider an example with six asserted triples maps over two CSV sources: `ratings.csv` and `users.csv`. `RatedMovieTM` produces the base quoted statement from `ratings.csv`, while `RatingAuditTM` consumes that quoted result on the same source. In parallel, `UserProfileTM` is an independent asserted triples map over `users.csv`, and `OccupationSummaryTM` re-

Planning Inputs**OTML Source Ordering**

Source Type	Source	Triples maps in source order
csv	ratings.csv	[RatedMovieTM, RatingAuditTM, UserOccupationProjectionTM, AuditInUserOccupationContextTM]
csv	users.csv	[UserProfileTM, OccupationSummaryTM]

Quoted Dependency Level Ordering

Triples map	Source	Level
RatedMovieTM	ratings.csv	0
UserProfileTM	users.csv	0
RatingAuditTM	ratings.csv	1
OccupationSummaryTM	users.csv	1
UserOccupationProjectionTM	ratings.csv	2
AuditInUserOccupationContextTM	ratings.csv	3



Level	Source	Triples maps executed in order
0	ratings.csv	[RatedMovieTM]
0	users.csv	[UserProfileTM]
1	ratings.csv	[RatingAuditTM]
1	users.csv	[OccupationSummaryTM]
2	ratings.csv	[UserOccupationProjectionTM]
3	ratings.csv	[AuditInUserOccupationContextTM]

Figure 5.4: Planning views used to derive the final quoted execution order. The planner combines preserved OTML-style source ordering with quoted dependency levels to produce a level-aware execution schedule.

mains source-local at the next level. `UserOccupationProjectionTM` depends on the quoted rating context, and `AuditInUserOccupationContextTM` consumes quoted results derived at the previous level. Figure 5.4 illustrates how the planner combines preserved OTML-style source ordering with quoted dependency levels to derive the final execution order.

5.2.3 Summary of Implementation Changes in TMP

In summary, the unified TMP phase preserves the original source-oriented planning structure of `SDM-RDFizer`, but extends it with explicit metadata for quoted dependencies, statement-level equivalence, and cache lifetime. The QDT captures quoted producer-consumer relations and dependency levels, the QET records support-to-owner statement equivalences, and the QFT derives flush boundaries for quoted intermediate results. The resulting execution plan is therefore a level-aware refinement of OTML: asserted triples maps are ordered by dependency level first and by source within each level, while non-asserted support mappings remain indirectly governed through the planning metadata rather than through direct insertion into the main execution schedule.

5.3 Execution Phase: From Triple Generation to Quoted-Result Reuse

After the planning phase has produced the ordered runtime schedule and the associated star-aware metadata, execution begins the actual materialization of RDF triples. In the unified implementation, asserted triples maps form the explicit top-level runtime schedule, whereas non-asserted support mappings are not inserted into that schedule as independent execution units and remain available only through lazy support-side materialization when needed. The execution phase therefore extends the runtime of `SDM-RDFizer` only where quoted dependencies, statement-level equivalence, and cache-lifetime constraints require additional support.

5.3.1 Preserved Execution Structures and Operators

Execution still proceeds by iterating over the relevant data sources, reading their rows, and materializing triples according to the mapping rules of each asserted triples map. Thus, the row-driven triple generation process of the base engine remains the operational foundation of the unified implementation.

This also applies to the basic execution-side structures described for the base engine. The Dictionary Table (DT) is still used to normalize and compactly represent runtime terms for duplicate-aware handling. Likewise, the Predicate Tuple Table (PTT) and Predicate Join Tuple Table (PJTT) continue to support the organization of predicate-oriented runtime information needed by the existing semantification logic. These structures therefore remain part of the execution foundation on top of which the unified extensions operate.

At the operator level, the original execution semantics are likewise retained. Simple Object Maps (SOMs) still generate object values directly from constants, templates, or references without requiring parent-side lookup. Object Reference Maps (ORMs) continue to resolve object values by directly referring to source attributes. Object Join Maps (OJMs) still realize parent-child joins by using join conditions between mappings. Thus, the core triple-construction behavior of the base engine remains intact.

What changes in the unified execution phase is not the existence of these structures and operators, but the runtime environment in which they operate. When quoted triples, cross-source quoted joins, or support-to-owner statement equivalences are involved, execution is augmented with additional mechanisms for quoted-result reuse and controlled cache lifetime. The next subsection introduces these unified execution-side data structures.

5.3.2 Unified Execution Data Structures

The unified contribution at execution time is centered on two coordinated row-keyed payload stores together with two join-keyed lookup structures. The first payload store is a unified cache for exact quoted producer-row payloads. The second payload store is an equivalence-aware cache for canonical statement-level payloads under owner-side identities. In addition, the preserved join table acquires a quoted execution role, and the equivalence-aware join structure exposes the same canonical statement slice in join-keyed form. Together, these structures make it possible to retain quoted results once they have been materialized, access them again through different runtime paths, and release them later according to the flush boundaries derived during planning.

Unified cache.

The unified cache stores the exact quoted payload produced by one producer triples map for one exact producer row. Its purpose is to decouple quoted-result reuse from recursive re-materialization. Conceptually, each cache entry is keyed by the producer triples map identifier together with a runtime row token of the form

$(producer\ TM\ id, (source\ name, row\ position)).$

The value stored under this key is the complete quoted payload materialized for that producer row. Thus, the unified cache does not store a join answer or a partially interpreted structure, but the exact row-level quoted result itself.

To illustrate the execution-side data structures, consider **RatedMovieTM** over **ratings.csv**. Assume the source contains the following rows:

userId	movieId	rating
u7	m42	4
u9	m42	5

Assume further that **RatedMovieTM** produces quoted rating statements for these rows. The following tables then illustrate how the unified runtime structures would store the corresponding entries.

Table 5.6: Conceptual unified-cache entries for **RatedMovieTM**.

Key	Entry
("RatedMovieTM", ("ratings.csv", 0))	[« user/u7 rated movie/m42 » ex:ratingValue "4" .]
("RatedMovieTM", ("ratings.csv", 1))	[« user/u9 rated movie/m42 » ex:ratingValue "5" .]

The unified cache is filled in two main situations. First, it is populated on same-row quoted access, when a quoted producer is requested from the same row context and the runtime already has a safe row token. In that case, execution either reuses the cached payload immediately or materializes it once and stores it for later accesses. Second, it is populated during quoted cross-source join-index construction, where producer rows are materialized once while the join index is being built. In that case, the join structure keeps the producer-row identities, while the actual quoted payload is stored separately in the unified cache.

When planning has already identified that a producer is consumed through both same-row and cross-source access, a safe top-level row scan can also retain the exact row payload in the unified cache while simultaneously staging the corresponding quoted join-index entries for the planner-known join signatures of that same producer. Thus, same-row-first execution can prepare part of the cross-source lookup path instead of forcing a later producer rescan.

Predicate Join Tuple Table (PJTT) in quoted execution.

The PJTT is preserved from the base engine, but its runtime interpretation depends on the execution path. In the ordinary non-star case, a complete PJTT lookup may be described conceptually as

$$((producer\ TM\ id, join\ field), join\ value) \rightarrow \{resolved\ RDF\ term : "object"\}.$$

Here, the PJTT stores the resolved parent-side payload directly under the corresponding join value.

In the quoted cross-source case, however, the PJTT is used differently. A complete PJTT lookup may then be described conceptually as

$$((quoted, producer\ TM\ id, join\ field), join\ value) \rightarrow [(source\ name, row\ position), \dots].$$

Thus, the PJTT no longer stores the final quoted payload itself. Instead, it stores the producer-row identities that satisfy the join value, while the actual quoted payload is retrieved later from the **unified cache**.

Assume that `RatedMovieTM` reads from `ratings.csv` and is later quoted from `movies.csv` through a join on `movieId`. Consider the following producer rows from `ratings.csv`:

userId	movieId	rating	timestamp
196	242	3	881250949
186	302	3	891717742

The corresponding quoted PJTT entries may then be written as follows:

Table 5.7: Conceptual PJTT entries for quoted cross-source access from `movies.csv` to `RatedMovieTM` through `movieId`.

Key	Entry
<code>(("quoted", "RatedMovieTM", "movieId"), "242")</code>	<code>[("ratings.csv", 0)]</code>
<code>(("quoted", "RatedMovieTM", "movieId"), "302")</code>	<code>[("ratings.csv", 1)]</code>

Thus, the quoted join path is split into two layers: the PJTT identifies which producer rows match a join value, and the unified cache stores what those producer rows actually produced.

Equivalence-aware reuse.

The second new execution-side reuse layer is the **equivalence-aware** layer. Its purpose is to support **Equivalence** cases as defined earlier in this work, that is, planner-detected cases in which a non-asserted support mapping and a predicate-object-map slice of a Multi-POM-TM denote the same statement-level payload. In the current implementation, this statement-level equivalence is represented canonically through the owner-side triples map and the corresponding owner predicate-object-map slice.

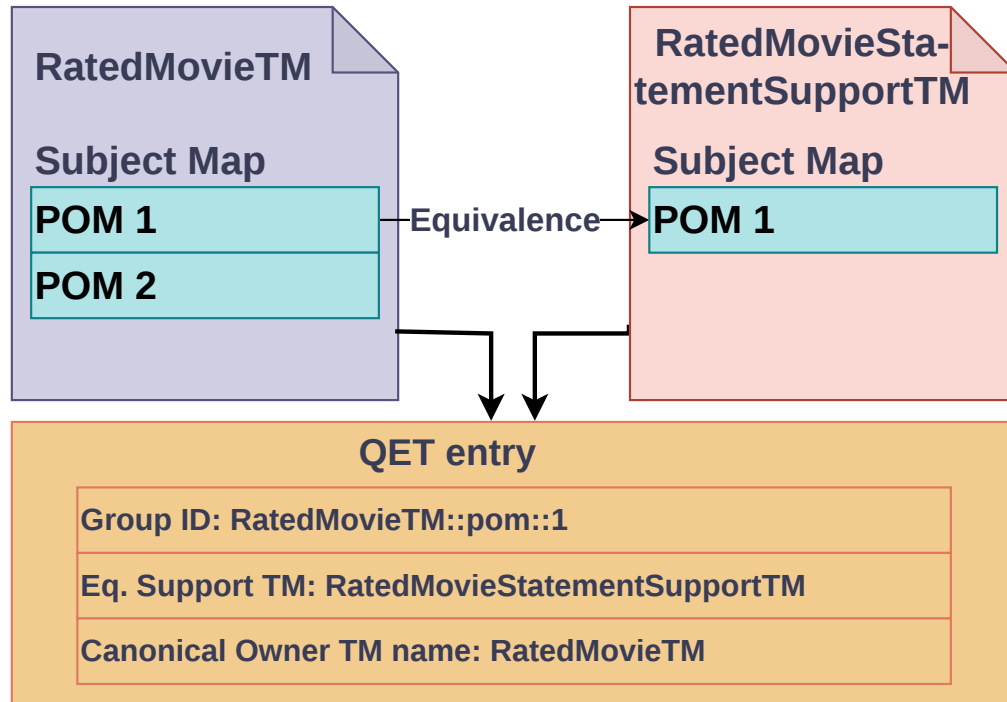


Figure 5.5: Equivalence view underlying the equivalence-aware reuse layer. The QET records that a non-asserted support triples map and a canonical owner-side predicate-object-map slice denote the same statement-level payload.

Figure 5.5 shows the equivalence view that underlies this reuse layer. The QET records the relation between the non-asserted support map, the canonical owner triples map, and the owner-side predicate-object-map slice that is treated as the canonical statement-level representative. This canonical owner-side identity is then used as the basis for equivalence-aware reuse during execution.

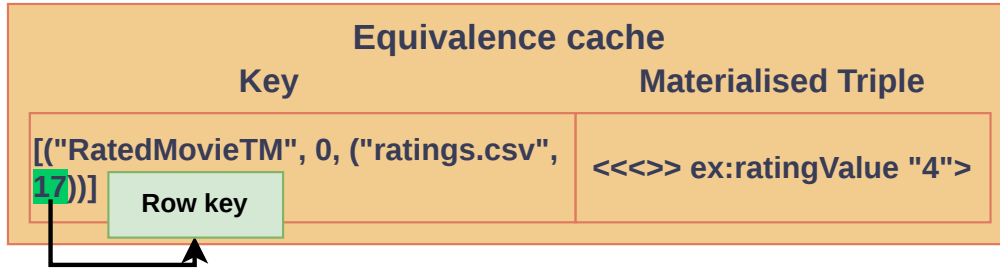


Figure 5.6: Same-row access to the equivalence cache. The cached statement-level payload is reused directly under the canonical owner-side equivalence identity for one exact row token.

Equivalence cache.

The first execution-side structure of this layer is the row-keyed **equivalence cache**. It stores the canonical owner-side statement-level payload for one exact row. Conceptually, each entry is keyed by the canonical owner triples map identifier, the canonical owner predicate-object-map index, and the runtime row token:

$$(\text{owner TM id}, \text{owner POM index}, (\text{source name}, \text{row position}))$$

The value stored under this key is the canonical statement-level payload materialized for that exact row. Thus, the equivalence cache is the payload store of the equivalence-aware reuse layer.

Figure 5.6 illustrates the same-row access case. Here the cached equivalence payload is reused directly for one exact row token. This corresponds to row-local reuse, where execution can retrieve the previously materialized canonical statement-level payload from the equivalence cache without requiring a join-based lookup.

Equivalence join table.

The second execution-side structure of this layer is the **equivalence join table**. It is a join-keyed lookup structure that records which canonical row entries are reachable through a given join value. Conceptually, it may be viewed as

$$((\text{owner TM id}, \text{owner POM index}, \text{join field}), \text{join value}) \rightarrow [(\text{source name}, \text{row position}), \dots]$$

Thus, the equivalence join table does not store a second full copy of the statement-level payload. Instead, it stores the canonical row tokens through which the corresponding payload can later be recovered from the row-keyed equivalence cache.

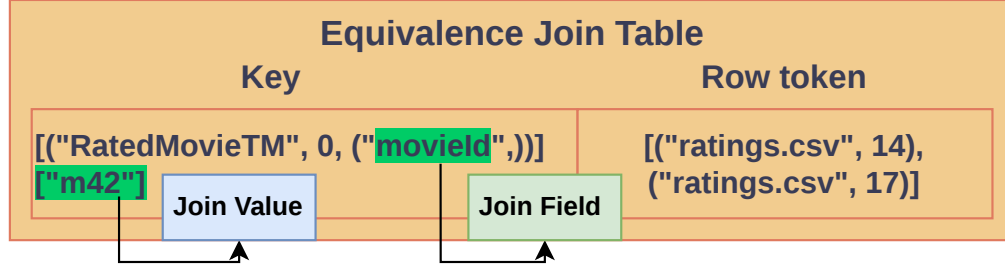


Figure 5.7: Cross-source join access to the equivalence-aware reuse layer. The equivalence join table identifies canonical row tokens, which are then resolved through the row-keyed equivalence cache.

Figure 5.7 illustrates the cross-source or join-keyed access case. Here the same canonical statement-level payload is not retrieved as a second independently stored payload. Instead, execution first uses the equivalence join table to identify the matching canonical row tokens, and then resolves those row tokens through the row-keyed equivalence cache to recover the corresponding statement-level payload. This access form becomes relevant when the quoted payload must be reached from a different source context or through a join signature rather than through direct row-local reuse.

When a safe full-row scan materializes a canonical equivalence payload, execution can also retain the row-keyed payload in the equivalence cache and simultaneously stage join-keyed lookup entries in the equivalence join table for the planner-known cross-source join signatures of the same equivalence group. Thus, later cross-source equivalence access can reuse the already scanned rows through a join-keyed token view instead of reconstructing the path from scratch.

In this way, the equivalence-aware reuse layer provides one canonical statement-level reuse model while allowing two operational access modes: same-row statement reuse through the equivalence cache and cross-source join-based statement reuse through the equivalence join table.

5.3.3 Base Operators in Unified Execution

Preserved base-engine step

Unified execution augmentation

The unified execution phase preserves the original SOM, ORM, and OJM operators as the construction kernels of SDM-RDFizer. The row-level execution order used to invoke these kernels follows the planned runtime schedule established by the planning phase. On top of this preserved execution foundation, the unified implementation augments runtime behavior with explicit quoted-result reuse, equivalence-aware

reuse, row-keyed payload retention, and join-keyed access structures. In the algorithms below, reuse structures are consulted only for lookup and recovery. Retention occurs only after a payload has actually been materialized.

Operator notation.

In the algorithms below, a *payload* denotes the materialized RDF-star output reused by the runtime. More precisely, let

$$\tau = (\textit{source name}, \textit{row position})$$

denote a row token,

$$P_{\text{row}}(tm, \tau)$$

the full row-level payload produced by triples map tm for row token τ ,

$$P_{\text{slice}}(\kappa, \tau)$$

the canonical statement-slice payload stored under canonical equivalence identity κ for row token τ ,

$$P_{\text{join}}(tm, j, val)$$

the quoted join payload assembled for triples map tm , join signature j , and join value val from the producer-row tokens stored in the quoted PJTT, and

$$P_{\text{eq}}(\kappa, j, val)$$

the equivalence-aware join payload assembled for canonical identity κ , join signature j , and join value val from the canonical row tokens exposed through the equivalence join table.

Moreover, let

$$U[(tm, \tau)] = P_{\text{row}}(tm, \tau)$$

denote the unified-cache entry for an exact producer-row payload,

$$E[(\kappa, \tau)] = P_{\text{slice}}(\kappa, \tau)$$

the equivalence-cache entry for one canonical statement slice on one exact row,

$$E^J[(\kappa, j, val)] = Tok_{\text{eq}}(\kappa, j, val)$$

the equivalence join-table entry that maps join signature j and join value val to the canonical row tokens through which the statement-level payload can later be recovered from E , and

$$PJTT[((quoted, tm, j), val)] = Tok(tm, j, val)$$

the quoted PJTT lookup that maps join value val to the matching producer-row token set.

Thus, U and E are row-keyed payload stores, whereas $PJTT$ and E^J are join-keyed lookup structures. Cross-source access does not retrieve payload directly from $PJTT$ or E^J . Instead, these structures identify the relevant row tokens, after which payload is resolved through U or E , respectively.

A triples map is called *quote-eligible* when its produced row-level payload must later remain available for quoted reuse in the unified cache. A produced statement slice is called *equivalence-eligible* when it belongs to a planner-detected canonical equivalence group. Produced triples are added to the KG only when the currently materialized triples map is asserted. If the triples map is non-asserted, its produced output is materialized only as reusable runtime payload and is not emitted to the KG as independent top-level output.

The row-local payload is deliberately separated from duplicate-suppressed KG emission. In particular, $Q_{tm, \tau}$ records the statement payload produced for the current row so that it can be reused by later quoted consumers, even when an equivalent subject-object combination has already been registered in the Predicate Tuple Table. The PTT therefore controls whether an asserted triple is emitted again to the final KG; it does not decide whether the corresponding quoted payload exists for reuse. This separation is necessary because quoted-result consumers may still require a payload that would be duplicate-suppressed from the ordinary asserted output path.

The notation $Subj(tm, row, \tau)$ and $Obj(po, row, \tau)$ denotes the existing subject- and object-resolution dispatch used by the materializer for the current row. For ordinary maps, these functions return the template, reference, constant, or parent-join values resolved from the current row. If the subject map or object map is a quoted triples map, the same dispatcher invokes the corresponding quoted-access procedure. The quoted access is therefore determined by the placement of `rml:quotedTriplesMap`: quoted subject placement is handled by subject resolution, while quoted object placement is handled by object resolution. Within either placement, the dispatcher chooses same-row access when producer and consumer share a logical source, and join-keyed access when the quoted producer must be reached through a join signature. For join-based object resolution, the dispatcher may receive the join signature j and current child-side join value val as additional arguments, written as

$Obj(po, row, \tau, j, val)$.

Preserved base operators under unified execution.

The base construction operators of SDM-RDFizer remain unchanged in their core triple-construction logic. Under unified execution, however, each preserved operator is augmented with post-production reuse management. After the current row has been materialized, its produced payload may be retained in the unified cache, retained in the equivalence cache, and, when the planner-derived access profile requires it, exposed through staged join-keyed access entries for later cross-source reuse.

Algorithm 5.1 Simple object map in unified execution (SOM-U)

Input: triples map tm_1 , current row row , row token τ , and a predicate-object map po whose predicate is p and whose simple object map is $f_2(att_2)$

Output: row-local payload $Q_{tm_1, \tau}$ and RDF triples emitted to the KG if tm_1 is asserted

- 1: Invoke subject resolution for (tm_1, row, τ) and let the result be $Subj(tm_1, row, \tau)$
If the subject map of tm_1 is quoted, this resolution invokes the same-row or cross-source quoted-access path for the current row token.
- 2: Resolve the simple object term $o \leftarrow f_2(row.att_2)$
- 3: Initialize the row-local payload $Q_{tm_1, \tau} \leftarrow []$
- 4: Let κ be the canonical identity of the current produced statement slice if eligible
- 5: **for all** $s \in Subj(tm_1, row, \tau)$ **do**
- 6: Create RDF triple $t = (s, p, o)$
- 7: Add t to the row-local reusable payload $Q_{tm_1, \tau}$
- 8: Add s and o to the Dictionary Table
- 9: **if** tm_1 is asserted and $encode(s, o)$ does not belong to the PTT for p **then**
- 10: Add $encode(s, o)$ to the PTT for p
- 11: Add t to the KG
- 12: **end if**
- 13: **end for**
- 14: Invoke Algorithm 5.8 for $(tm_1, \tau, Q_{tm_1, \tau}, \alpha(tm_1), \Sigma(tm_1))$
- 15: Invoke Algorithm 5.9 for $(\kappa, \tau, Q_{tm_1, \tau}, \Sigma(\kappa))$
- 16: **return** $Q_{tm_1, \tau}$

Algorithm 5.2 Object reference map in unified execution (ORM-U)

Input: triples map tm_1 , current row row , row token τ , and a predicate-object map po whose predicate is p and whose object reference is resolved as $f_2(att_2)$

Output: row-local payload $Q_{tm_1,\tau}$ and RDF triples emitted to the KG if tm_1 is asserted

- 1: Invoke subject resolution for (tm_1, row, τ) and let the result be $Subj(tm_1, row, \tau)$
If the subject map of tm_1 is quoted, this resolution returns quoted subject payloads for the current row by same-row or cross-source access.
- 2: Invoke object resolution for $(po.objectMap, row, \tau)$ and let the result be $Obj(po, row, \tau)$
For a preserved object reference map this is the singleton $\{f_2(row.att_2)\}$; if the object map is quoted, this resolution invokes the quoted-object access path for the current row token.
- 3: Initialize the row-local payload $Q_{tm_1,\tau} \leftarrow []$
- 4: Let κ be the canonical identity of the current produced statement slice if eligible
- 5: **for all** $s \in Subj(tm_1, row, \tau)$ **do**
- 6: **for all** $o \in Obj(po, row, \tau)$ **do**
- 7: Create RDF triple $t = (s, p, o)$
- 8: Add t to the row-local reusable payload $Q_{tm_1,\tau}$
- 9: Add s and o to the Dictionary Table
- 10: **if** tm_1 is asserted and $encode(s, o)$ does not belong to the PTT for p **then**
- 11: Add $encode(s, o)$ to the PTT for p
- 12: Add t to the KG
- 13: **end if**
- 14: **end for**
- 15: **end for**
- 16: Invoke Algorithm 5.8 for $(tm_1, \tau, Q_{tm_1,\tau}, \alpha(tm_1), \Sigma(tm_1))$
- 17: Invoke Algorithm 5.9 for $(\kappa, \tau, Q_{tm_1,\tau}, \Sigma(\kappa))$
- 18: **return** $Q_{tm_1,\tau}$

Algorithm 5.3 Object join map in unified execution (OJM-U)

Input: triples map tm_1 , current row $row_1 \in S_1$, row token τ_1 , and a predicate-object map po whose predicate is p and whose object map is resolved through join signature j

Output: row-local payload Q_{tm_1, τ_1} and RDF triples emitted to the KG if tm_1 is asserted

- 1: Invoke subject resolution for (tm_1, row_1, τ_1) and let the result be $Subj(tm_1, row_1, \tau_1)$
If the subject map of tm_1 is quoted, this resolution returns quoted subject payloads for the current row token.
- 2: Resolve the child-side join value val of row_1 under join signature j
- 3: Invoke object resolution for $(po.objectMap, row_1, \tau_1, j, val)$ and let the result be
- 4: $Obj(po, row_1, \tau_1)$
If $po.objectMap$ is a parent triples map, object resolution uses the preserved PJTT parent lookup; if it is a quoted triples map, object resolution invokes the cross-source quoted-access
- 5: path for (j, val) .
- 6: Initialize the row-local payload $Q_{tm_1, \tau_1} \leftarrow []$
- 7: Let κ be the canonical identity of the current produced statement slice if eligible
- 8: **for all** $s \in Subj(tm_1, row_1, \tau_1)$ **do**
- 9: **for all** $o \in Obj(po, row_1, \tau_1)$ **do**
- 10: Create RDF triple $t = (s, p, o)$
- 11: Add t to the row-local reusable payload Q_{tm_1, τ_1}
- 12: Add s and o to the Dictionary Table
- 13: **if** tm_1 is asserted and $encode(s, o)$ does not belong to the PTT for p **then**
- 14: Add $encode(s, o)$ to the PTT for p
- 15: Add t to the KG
- 16: **end if**
- 17: **end for**
- 18: **end for**
- 19: Invoke Algorithm 5.8 for $(tm_1, \tau_1, Q_{tm_1, \tau_1}, \alpha(tm_1), \Sigma(tm_1))$
- 20: Invoke Algorithm 5.9 for $(\kappa, \tau_1, Q_{tm_1, \tau_1}, \Sigma(\kappa))$
- 21: **return** Q_{tm_1, τ_1}

New quoted-access operators.

The main execution-side novelty lies not in replacing the preserved triple-construction kernels, but in the quoted-access procedures invoked by subject and object resolution. These procedures are row-local from the consumer's point of view: they receive the current consumer row token, recover or materialize the quoted producer payload needed for that row, and return that payload to the enclosing SOM-U, ORM-U, or OJM-U operator. Emission to the KG and retention of the consumer-row payload remain the responsibility of the enclosing operator.

Algorithm 5.4 Quoted same-row access in unified execution (QSRA)

Input: consumer triples map tm_c , producer triples map tm_p , current consumer row row_c , and row token τ_c , where tm_c and tm_p use the same logical source

Output: quoted producer payload for the current row token

- 1: Let κ_p be the canonical identity of the producer statement slice if eligible
- 2: Invoke Algorithm 5.6 for (κ_p, τ_c)
- 3: **if** a hit is returned as Q_p **then**
- 4: **return** Q_p
- 5: **end if**
- 6: Invoke Algorithm 5.7 for (tm_p, τ_c)
- 7: **if** a hit is returned as Q_p **then**
- 8: **return** Q_p
- 9: **end if**
- 10: Invoke the producer-side execution of tm_p for row_c under its own runtime path
- 11: Let Q_p be the row-local payload produced by tm_p for τ_c
- 12: Invoke Algorithm 5.8 for $(tm_p, \tau_c, Q_p, \alpha(tm_p), \Sigma(tm_p))$
- 13: Invoke Algorithm 5.9 for $(\kappa_p, \tau_c, Q_p, \Sigma(\kappa_p))$
- 14: **return** Q_p

Algorithm 5.5 Quoted cross-source access in unified execution (QCSA)

Input: producer triples map tm_p , producer source S_p , current consumer row token τ_c , join signature j , and current consumer-side join value val

Output: quoted producer payload assembled for the current consumer row

- 1: Treat τ_c as the enclosing consumer-row context; QCSA returns producer payload and does not emit or retain consumer output
- 2: Let κ_p be the canonical identity of the producer statement slice if eligible
- 3: Invoke Algorithm 5.11 for (κ_p, j, val)
- 4: **if** a hit is returned as $P_{eq}(\kappa_p, j, val)$ **then**
- 5: **return** $P_{eq}(\kappa_p, j, val)$
- 6: **end if**
- 7: **if** $PJTT[(quoted, tm_p, j), val]$ does not exist **then**
- 8: Invoke Algorithm 5.12 for (tm_p, S_p, j)
- 9: **end if**
- 10: **if** $PJTT[(quoted, tm_p, j), val]$ still does not exist **then**
- 11: **return** empty payload
- 12: **end if**
- 13: Invoke Algorithm 5.10 for (tm_p, j, val)
- 14: **if** a hit is returned as $P_{join}(tm_p, j, val)$ **then**
- 15: **return** $P_{join}(tm_p, j, val)$
- 16: **end if**
- 17: **return** empty payload

Helper algorithms.

To avoid repeating the same reuse-management steps across multiple operators, the unified implementation factors lookup, retention, join-view staging, and cross-source payload resolution into helper algorithms. These helpers do not introduce additional execution paths of their own. Instead, they encapsulate the common runtime actions reused by both the preserved base operators and the new quoted-access operators.

Lookup helpers

The lookup helpers perform the two row-keyed access checks used throughout unified execution. Algorithm 5.6 tests whether a canonical statement slice is already available in the equivalence cache, whereas Algorithm 5.7 tests whether an exact producer-row payload is already available in the unified cache.

Algorithm 5.6: Check row-keyed equivalence payload

Input: canonical identity κ and row token τ

Output: canonical statement-slice payload if available

```

1: if  $E[(\kappa, \tau)]$  exists then
2:   Return  $E[(\kappa, \tau)]$ 
3: end if
4: Return miss

```

Algorithm 5.7: Check unified row payload

Input: triples map tm and row token τ

Output: row-level quoted payload if available

```

1: if  $U[(tm, \tau)]$  exists then
2:   Return  $U[(tm, \tau)]$ 
3: end if
4: Return miss

```

Retention Helpers

The retention helpers are invoked only after a row payload has actually been materialized by the current operator or by a recursive producer-side execution path. They do not construct new RDF-star payload on their own; instead, they record already materialized payload in the runtime reuse structures. At the row-keyed level, they preserve the result under its exact row identity, either in the unified cache or in the equivalence cache, depending on the reuse model to which the payload belongs. When the planner-derived access profile further indicates that the same result may later be reached from a cross-source or mixed access path, the helpers also stage the corresponding join-keyed entries. Thus, these helpers serve as the bridge between

immediate row-local materialization and later reuse through either direct row-keyed lookup or join-based access. For a pure cross-source quoted access path, the quoted PJTT is prepared by the dedicated index-build helper. The retention helper stages quoted PJTT entries only for mixed access, where a row-local materialization can safely expose the same payload for a later join-keyed lookup.

Algorithm 5.8: Retain unified row payload and stage quoted join views

Input: triples map tm , row token τ , produced output payload Q and access profile $\alpha(tm)$ with join signatures $\Sigma(tm)$

Output: updated unified cache and staged quoted PJTT entries

```

1: Construct  $P_{\text{row}}(tm, \tau)$  from  $Q$ 
2: if  $tm$  is quote-eligible then
3:   Set  $U[(tm, \tau)] \leftarrow P_{\text{row}}(tm, \tau)$ 
4: end if
5: if  $\alpha(tm) \in \{\text{mixed}\}$  then
6:   for all  $j \in \Sigma(tm)$  do
7:     Resolve the join value  $val$  of the
       current row under  $j$ 
8:     if  $val$  is defined and
        $P_{\text{row}}(tm, \tau) \neq []$  then
9:       Stage  $\tau$  for
        $PJTT[((\text{quoted}, tm, j), val)]$ 
10:    end if
11:  end for
12: end if

```

Algorithm 5.9: Retain equivalence payload and stage equivalence join views

Input: canonical identity κ , row token τ , produced output payload Q and access profile for the equivalence group with join signatures $\Sigma(\kappa)$

Output: updated equivalence cache and staged equivalence join-table entries

```

1: if the produced statement slice is
   equivalence-eligible then
2:   Construct  $P_{\text{slice}}(\kappa, \tau)$  from  $Q$ 
3:   Set  $E[(\kappa, \tau)] \leftarrow P_{\text{slice}}(\kappa, \tau)$ 
4:   if the equivalence-group access profile
     requires cross-source access then
5:     for all  $j \in \Sigma(\kappa)$  do
6:       Resolve the join value  $val$  of the
       current row under  $j$ 
7:       if
          $val$  is defined and  $P_{\text{slice}}(\kappa, \tau) \neq []$  then
8:         Stage  $\tau$  for  $E^J[(\kappa, j, val)]$ 
9:       end if
10:    end for
11:  end if
12: end if

```

The cross-source resolution helpers make the two-step nature of join-based reuse explicit. They first obtain row tokens from a join-keyed access structure and then resolve those tokens through the corresponding row-keyed payload store.

Algorithm 5.10: Resolve quoted cross-source join payload

Input: producer triples map tm_p , join signature j , and join value val

Output: assembled quoted join payload if available

```

1: if  $PJTT[((quoted, tm_p, j), val)]$  does not exist then
2:   Return miss
3: end if
4: Let  $Tok(tm_p, j, val) \leftarrow PJTT[((quoted, tm_p, j), val)]$ 
5: Initialize  $P_{join}(tm_p, j, val) \leftarrow []$ 
6: Let  $\kappa_p$  be the canonical identity of the producer statement slice if eligible
7: for all  $\tau_p \in Tok(tm_p, j, val)$  do
8:   Invoke Alg. 5.7 for  $(tm_p, \tau_p)$ 
9:   if a hit is returned as  $Q_p$  then
10:    Add  $Q_p$  to  $P_{join}(tm_p, j, val)$ 
11:   else
12:     Invoke the producer-side execution of  $tm_p$  for the row identified by  $\tau_p$  under its own runtime path
13:   Let  $Q_p$  be the payload returned by this recursive producer-side execution
14:   Invoke Alg. 5.8 for  $(tm_p, \tau_p, Q_p, \alpha(tm_p), \Sigma(tm_p))$ 
15:   Invoke Alg. 5.9 for  $(\kappa_p, \tau_p, Q_p, \Sigma(\kappa_p))$ 
16:   Add  $Q_p$  to  $P_{join}(tm_p, j, val)$ 
17: end if
18: end for
19: Return  $P_{join}(tm_p, j, val)$ 

```

Algorithm 5.11: Resolve equivalence cross-source join payload

Input: canonical identity κ , join signature j , and join value val

Output: assembled equivalence-aware join payload if available

```

1: if  $E^J[(\kappa, j, val)]$  does not exist then
2:   Return miss
3: end if
4: Let  $Tok_{eq}(\kappa, j, val) \leftarrow E^J[(\kappa, j, val)]$ 
5: Initialize  $P_{eq}(\kappa, j, val) \leftarrow []$ 
6: for all  $\tau \in Tok_{eq}(\kappa, j, val)$  do
7:   Invoke Algorithm 5.6 for  $(\kappa, \tau)$ 
8:   if a hit is returned as  $Q_{eq}$  then
9:     Add  $Q_{eq}$  to  $P_{eq}(\kappa, j, val)$ 
10:  else
11:    Return miss
12:  end if
13: end for
14: Return  $P_{eq}(\kappa, j, val)$ 

```

PJTT Index build Helper

The index-build helper captures the explicit producer-side construction path used for quoted join indexing. It is invoked when cross-source quoted access requires a quoted PJTT entry that is not yet available in the current runtime state. In that case, the helper scans the relevant producer-side rows, reuses already available row payload where possible, and otherwise invokes the producer-side execution path for the missing rows. The resulting row tokens are then registered under the corresponding quoted PJTT join entries.

Algorithm 5.12 Build quoted PJTT index (Helper algorithm)

Input: producer triples map tm_p , source S_p , and join signature j

Output: quoted PJTT entries for tm_p on j

- 1: Let κ_p be the canonical identity of the producer statement slice if eligible
- 2: **for all** $row_p \in S_p$ **do**
- 3: Let τ_p be the row token of row_p
- 4: Invoke Algorithm 5.7 for (tm_p, τ_p)
- 5: **if** a hit is returned as Q_p **then**
- 6: Reuse Q_p as the producer-row payload for τ_p
- 7: **else**
- 8: Invoke the producer-side execution of tm_p for row_p under its own
- 9: runtime path
- 10: Let Q_p be the payload returned by this recursive producer-side
- 11: execution
- 12: Invoke Algorithm 5.8 for $(tm_p, \tau_p, Q_p, \alpha(tm_p), \Sigma(tm_p))$
- 13: Invoke Algorithm 5.9 for $(\kappa_p, \tau_p, Q_p, \Sigma(\kappa_p))$
- 14: **end if**
- 15: Let val be the join value of row_p under j
- 16: **if** $Q_p \neq []$ **then**
- 17: Add τ_p to $PJTT[((quoted, tm_p, j), val)]$
- 18: **end if**
- 19: **end for**

5.4 Summary

This chapter described the implementation of unified quoted execution in SDM-RDFizer. It first introduced the execution-side data structures that support reuse, namely the unified cache, the quoted PJTT, the equivalence cache, and the equivalence join table. It then showed how the preserved base operators are augmented with payload retention and staged join-keyed access, while new quoted-access operators govern same-row and cross-source reuse. Finally, the helper algorithms were presented as the common runtime mechanisms for row-keyed lookup, payload retention, join-view staging, and cross-source payload resolution. Taken together, these implementation components realize the planned separation between row-local materialization, reusable payload retention, and join-based access to previously materialized quoted results.

Chapter 6

Experimental Evaluation

6.1 Evaluation Goals

The evaluation in this thesis is correctness-first and memory-centered. It addresses three questions. First, does the unified execution strategy preserve the intended RDF-star result on layered mappings that reuse quoted statements through same-source and cross-source access paths? Second, after correctness is established, how does peak memory behave as the mapping structure becomes deeper and broader? Third, how do the observed behaviors differ between SDM-RDFizer-Plus and Morph-KGC on the same mapping families?

Runtime is still reported, but it is not treated as the main comparison criterion. This is deliberate. Morph-KGC is designed around mapping partitions, which group compatible mapping rules and can reduce repeated scans and intermediate work; this makes runtime a natural strength of Morph-KGC and is consistent with the motivation of the Morph-KGC paper [3]. In this benchmark, however, the main research question is whether the unified SDM-RDFizer-style implementation can retain complete quoted results while controlling the memory cost of that retention. For that reason, the discussion uses runtime mainly to explain how quickly a memory blow-up occurs, while the central metric is peak memory.

The evaluation therefore follows two admissibility rules. First, an engine run is eligible for performance comparison only if the corresponding output is correct for the benchmark configuration. Second, measurements from runs marked `killed_memory_limit` or `failed` are not interpreted as ordinary runtime or memory measurements. A killed run records that the process reached the configured memory boundary; its time is a truncated time-to-kill, and its memory is evidence of memory-limit pressure rather than a successful peak memory value. A failed run is even less comparable:

both its runtime and memory may reflect an early parser, planner, or execution error rather than materialization work.

6.2 Experimental Setup

6.2.1 Data Sources and Dataset Sizes

All benchmark configurations use the same three CSV logical sources: `ratings.csv`, `users.csv`, and `movies.csv`. The data are derived from the MovieLens source family introduced in Chapter [1](#). The benchmark keeps the logical-source schemas fixed while varying only the mapping structure and dataset size.

The correctness gate uses a `3line` dataset, a tiny slice of the same three logical sources with one header row and three data rows per file. This dataset is small enough for direct inspection, but still large enough to trigger the joins used by the benchmark. The scaling runs use 10000, 100000, and 500000. The 500000 instance was created as five identifier-disjoint copies of the 100000 instance, so the larger source preserves the same schema, mapping semantics, and join structure while avoiding accidental identifier collisions. The benchmark enforced a memory limit of 28 GB per run and a timeout of four hours (14 400 seconds) per run.

6.2.2 Engines and Measurement Scope

The evaluation considers three engines:

- the naive SDM-RDFizer implementation,
- SDM-RDFizer-Plus, the unified implementation proposed in this thesis,
- Morph-KGC 2.10.0.

The naive SDM-RDFizer implementation is included in the correctness discussion because it is the baseline from which the unified implementation was derived. It is not used as a fair runtime or memory baseline in the scaling comparison. The reason is not that the naive implementation is slow; in several small or simple cases it is fast and memory-light. The issue is that it can omit valid RDF-star output, produce malformed quoted results, or fail on the layered patterns used in this evaluation. Resource numbers from such runs are therefore not measuring the same completed task.

Consequently, the scaling comparison focuses on Morph-KGC and SDM-RDFizer-Plus. Even for these two engines, the graphs distinguish completed runs from killed and failed runs. Killed and failed values are shown visually because they explain

where the benchmark stopped being feasible, but they are not interpreted as successful performance points.

6.2.3 Execution Environment

The benchmark runs were executed on a desktop system with an Intel(R) Core(TM) i7-7800X CPU at 3.50 GHz, providing 6 physical cores and 12 logical threads, and with approximately 32 GB of RAM available to the operating system. The software environment was Debian GNU/Linux forkysid (64 bit) with Linux kernel 6.19.11+deb14+amd64. The machine also included an Intel Arc A750 graphics card with 8 GB VRAM; however, the evaluated engines are CPU-bound, so GPU acceleration was not part of the benchmark methodology.

6.3 Layered Configuration Set

The layered benchmark family is organized by depth and configuration number. Although the labels emphasize depth, the configurations are primarily designed as a quoted-result reuse stress ladder. Each configuration combines four aspects: the mapping operators used, the quoted access paths, the way quoted intermediate results are reused, and the marker predicates that reveal whether the final layer was materialized.

The reuse dimension is important because the main implementation problem is not only whether a quoted statement can be produced once. The harder question is whether the same quoted payload can remain available when it is consumed by several later mappings, reached through both same-source and cross-source paths, nested inside another quoted statement, or used simultaneously in quoted subject and quoted object position. The layered family therefore increases reuse pressure step by step.

6.3.1 Two-Layer Family

The two-layer family checks first-order quoted reuse. It starts with a single cross-source consumer, then introduces shared reuse of the same support slice, and finally increases fanout over the reused payload.

- `layer2_conf1`: one non-asserted support map plus one asserted consumer; cross-source quoted-subject join from `ratings.csv` to `movies.csv`. This is the minimal cross-source reuse case, where one quoted producer payload is reused by one

later consumer; markers `ex:movieTitleProjection`, `ex:movieReleaseProjection`, and `ex:movieActionFlag`.

- `layer2_conf2`: the same non-asserted base slice reused by one same-source quoted-subject audit consumer and two cross-source quoted-subject consumers toward `users.csv` and `movies.csv`; this tests shared reuse of one quoted payload through multiple access paths. Markers are `ex:auditTag`, `ex:occupationProjection`, and `ex:movieTitleProjection`.
- `layer2_conf3`: the same structure as `layer2_conf2` with additional movie-side fanout and more predicate-object maps; this keeps the shared producer fixed while increasing the number of outputs derived from the reused quoted context. Markers are `ex:movieComedyFlag`, `ex:movieThrillerFlag`, and `ex:movieImdbEcho`.

6.3.2 Three-Layer Family

The three-layer family checks nested quoted reuse. Here, quoted results are no longer only consumed as final inputs; they become intermediate contexts that are themselves quoted, reused, or fanned out by later mappings.

- `layer3_conf1`: a base quoted statement, a quoted audit support map, and one cross-source quoted-subject movie-context consumer over an already quoted statement. This tests reuse of a second-layer quoted payload rather than only reuse of the original base statement; markers `ex:movieTitleProjection`, `ex:movieReleaseProjection`, and `ex:movieDramaFlag`.
- `layer3_conf2`: cross-source quoted-subject movie reuse followed by same-source quoted-subject recursion over the already quoted movie-title slice. This combines join-keyed reuse with row-local recursive reuse; markers `ex:actionEcho`, `ex:titleLengthBand`, and `ex:releaseEcho`.
- `layer3_conf3`: an intermediate quoted audit slice fans out into parallel user-side and movie-side cross-source quoted-subject branches. This stresses reuse of the same intermediate quoted context by independent downstream branches; markers `ex:occupationProjection`, `ex:userAgeProjection`, `ex:genderProjection`, `ex:movieTitleProjection`, `ex:movieReleaseProjection`, `ex:movieDramaFlag`, and `ex:movieComedyFlag`.

6.3.3 Intermediate Mild-Recursive Control

This control case isolates a mild recursive reuse pattern between the three-layer and four-layer families. It checks whether the engine can reuse a quoted context directly while also resolving a separate cross-source quoted object.

- `mild_recursive_context`: a base quoted statement, a quoted audit statement, a cross-source user-context support statement, and an outer rule with direct quoted-subject reuse plus one cross-source quoted object join; marker `ex:reviewedInOccupationContext`.

6.3.4 Four-Layer Family

The four-layer family is the strongest reuse stress family. It combines deep quoted contexts, repeated reuse of already contextualized statements, cross-source joins, and in the hardest case simultaneous quoted subject and quoted object reuse.

- `layer4_conf1`: the first deep mixed case; same-source quoted-subject reuse over the audit slice plus cross-source quoted-object joins to deeper user-side and movie-side support slices. This tests whether a reused quoted subject can be combined with independently reused quoted objects; markers `ex:userContextEcho` and `ex:movieContextEcho`.
- `layer4_conf2`: the user-side and movie-side contexts from `layer4_conf1`, followed by further same-source quoted-subject recursion over the already contextualized user-side statement. This extends reuse lifetime because an already reused fourth-layer context is consumed again; markers `ex:finalConfidenceLiteral`, `ex:finalConfidenceBand`, and `ex:finalMovieEcho`.
- `layer4_conf3`: the strongest stress case; a final asserted rule combines a cross-source quoted-join subject path to the user source with a cross-source quoted-join object path to the movie source in the same outer rule. This is the highest reuse-pressure case because two independent cross-source quoted payloads must be retained and aligned in one output rule; markers `ex:crossSourceAlignment` and `ex:alignmentStamp`.

6.4 Correctness-Oriented Results

6.4.1 The `3line` Gate

Before any scaling measurement, all layered configurations were first run on the `3line` dataset. The gate verifies that each configuration is semantically reachable and makes omissions visible at the predicate level before larger outputs become impractical to inspect.

SDM-RDFizer-Plus completed all nine layered configurations on this correctness gate. The top-layer marker predicates were present even in the hardest cases, including `ex:userContextEcho` and `ex:movieContextEcho` for `layer4_conf1`, the final-confidence

predicate family for `layer4_conf2`, and `ex:crossSourceAlignment` for `layer4_conf3`. This establishes that the unified execution strategy can materialize the full layered family on the benchmark’s smallest instance.

The naive SDM-RDFizer implementation did not provide a reliable correctness baseline for these layered RML-star mappings. On the same correctness family, it either omitted expected triples, produced malformed quoted output, or failed on deeper dependency patterns. Therefore, its later time and memory values are not used as evidence of superior efficiency: the engine is often doing less work or producing a different output.

Morph-KGC behaved differently. It matched the expected output for the supported layered cases, but its known stress point is the final alignment-style pattern in `layer4_conf3`, where a quoted-join subject path and a quoted-join object path are combined in the same outer rule. In the scaling results, Morph-KGC also reaches the memory limit in several deep or reuse-heavy cases. For the `mild_recursive_context` control case, the updated measurements show a completed 100000 run with high peak memory and a 500000 run killed at the configured memory boundary. Therefore, the Morph-KGC 500000 time for this case is interpreted only as time-to-kill, not as completed materialization runtime.

6.4.2 Admissibility of Runtime and Memory Metrics

The correctness gate determines which later performance numbers are meaningful. If an engine omits the marker predicates of a configuration, then its runtime and memory are not comparable with an engine that produced the full expected output. This is why the naive SDM-RDFizer measurements are excluded from the main scaling comparison.

The same principle is applied to failed and killed runs of Morph-KGC and SDM-RDFizer-Plus. A `killed_memory_limit` point is useful because it shows that the run reached the configured memory boundary, but its runtime is only the time until termination and its memory is a boundary observation. A `failed` point is not a performance point at all; it may reflect an early mapping parser, partitioner, or execution exception. The graphs therefore show killed values with black dots over the engine color and failed values with stripes, making the disqualification visible in the figure itself.

6.5 Scaling Results

6.5.1 Reading the Figures

The scaling figures report all three measured engines: the naive SDM-RDFizer implementation, Morph-KGC, and SDM-RDFizer-Plus. The naive engine is included only as a diagnostic reference, because the correctness gate shows that it does not reliably materialize the full layered RML-star output. The main performance comparison therefore remains between Morph-KGC and SDM-RDFizer-Plus.

Both figures include 10000, 100000, and 500000. The memory axis uses a base-10 logarithmic scale in MB, because the benchmark ranges from small completed two-layer runs below 1 GB to deep runs near the configured memory boundary. The time figure also uses a base-10 logarithmic scale, because runtimes range from a few seconds to thousands of seconds. Completed bars are solid. Bars with black dots reached the memory limit. Striped bars correspond to failed runs. Killed and failed values are shown for diagnosis, but they should not be read as successful performance points.

6.5.2 Memory Behavior on Level 2 and Level 3

On the level-2 and level-3 configurations, both Morph-KGC and SDM-RDFizer-Plus complete the 100000 and 500000 runs. These are therefore the cleanest memory comparisons.

At 100000, SDM-RDFizer-Plus uses less memory than Morph-KGC in every level-2 and level-3 configuration. The reductions are 21.8% for L2c1, 13.4% for L2c2, 11.9% for L2c3, 3.7% for L3c1, 15.7% for L3c2, and 4.6% for L3c3. This should not be over-interpreted as a general language-level advantage, since both systems are Python-based. A more cautious interpretation is that Morph-KGC’s dataframe- and partition-oriented execution has a fixed overhead that is visible at this scale, whereas SDM-RDFizer-Plus stores only the quoted payloads and join views required by the unified cache model.

At 500000, the picture changes. SDM-RDFizer-Plus is slightly or moderately higher in five of the six level-2 and level-3 configurations: 2.4% higher for L2c1, 14.2% for L2c2, 11.9% for L2c3, 24.5% for L3c1, and 22.1% for L3c3. The exception is L3c2, where SDM-RDFizer-Plus is 5.1% lower than Morph-KGC. This supports a nuanced conclusion: the unified cache and equivalence-aware reuse do not create a dramatic memory penalty on the supported shallow and medium-depth cases, but their retained row-keyed payloads become more visible as the source grows.

Experimental Evaluation

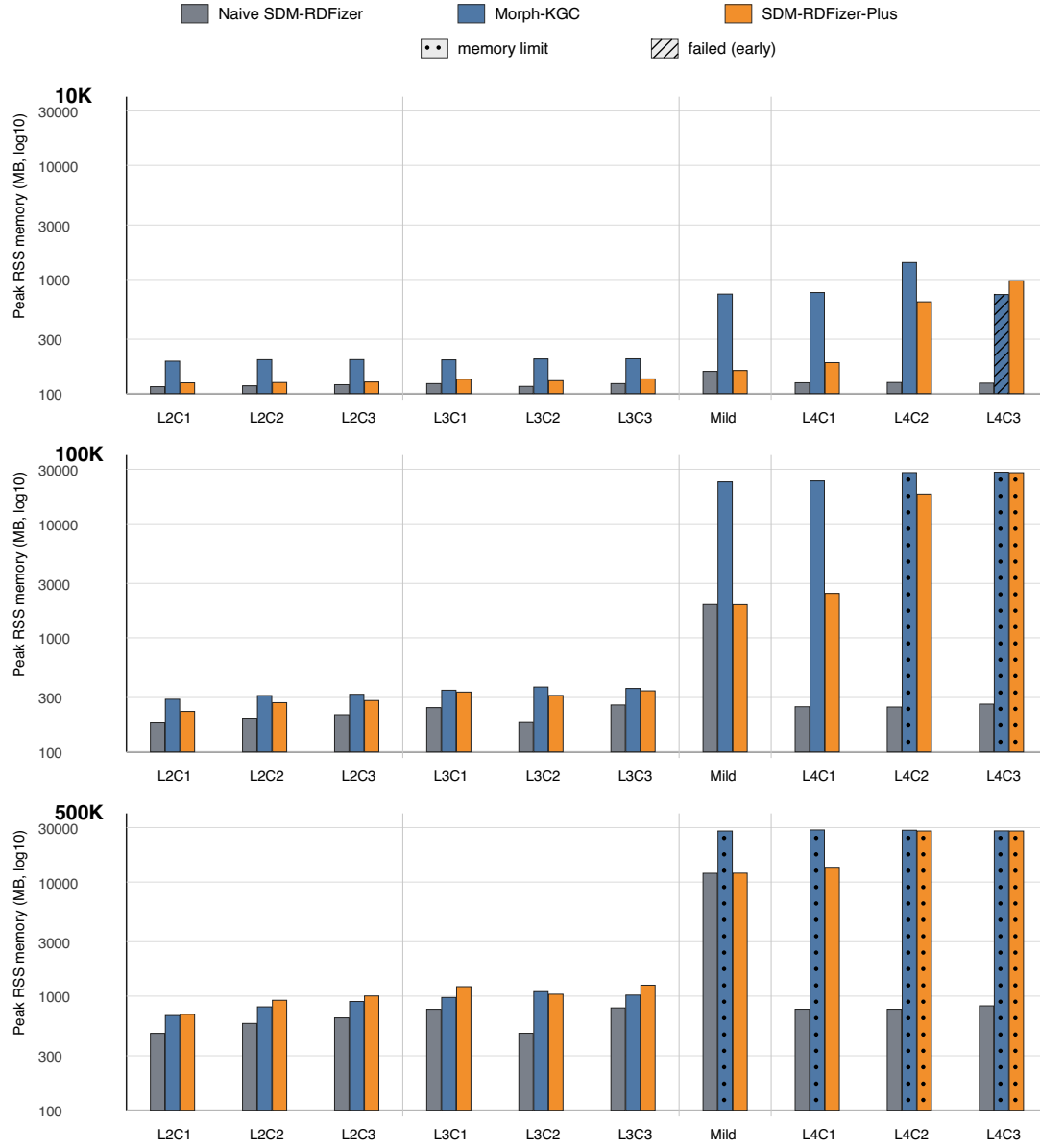


Figure 6.1: Peak RSS memory for the naive SDM-RDFizer implementation, Morph-KGC, and SDM-RDFizer-Plus on the 10000, 100000, and 500000 source runs. The memory axis is logarithmic base 10 in MB. Black dots mark memory-killed runs and diagonal stripes mark failed runs; these points are diagnostic and are not successful peak-memory measurements.

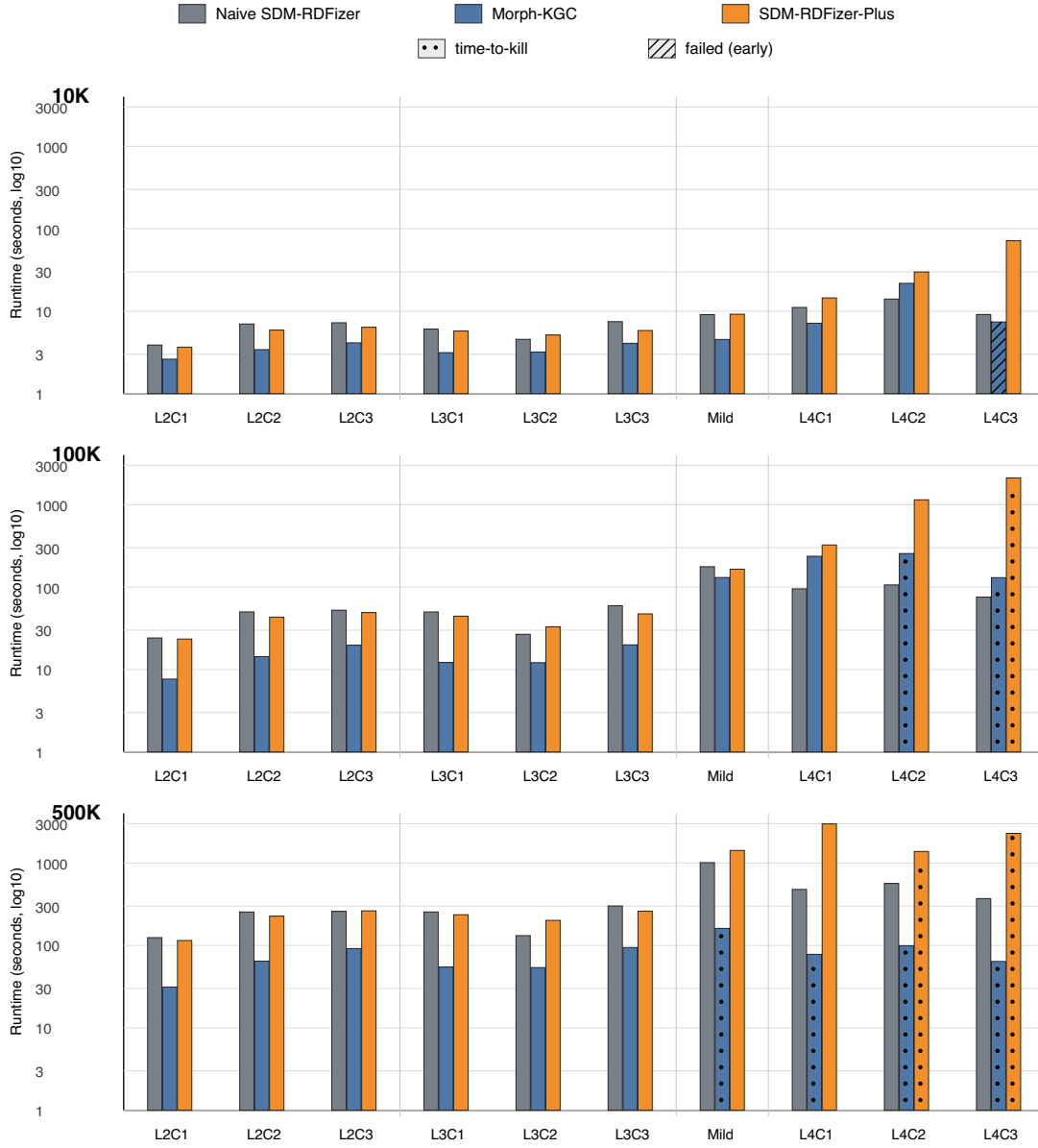


Figure 6.2: Runtime for the naive SDM-RDFizer implementation, Morph-KGC, and SDM-RDFizer-Plus on the 10000, 100000, and 500000 source runs. The time axis is logarithmic base 10 in seconds. For memory-killed runs, the shown value is time-to-kill; for failed runs, the shown value is not a completed materialization time.

The growth factors from 100000 to 500000 confirm this. The input size grows by 5x. Across L2C1–L3C3, SDM-RDFizer-Plus memory grows by about 3.06x to 3.63x, while its runtime grows by about 4.89x to 6.10x. Morph-KGC memory grows by about 2.34x to 2.95x, while its runtime grows by about 4.08x to 4.73x. Thus, Morph-KGC shows the stronger time behavior on these supported cases, which is consistent with its partitioned execution design, while SDM-RDFizer-Plus remains within the expected scaling neighborhood for a cache-retaining implementation.

6.5.3 Deep Configurations and Memory-Limit Behavior

The deeper configurations are where the engines diverge most strongly. At 10000, all three engines complete `mild_recursive_context`. SDM-RDFizer-Plus and the naive SDM-RDFizer implementation both remain below 160 MB peak RSS, while Morph-KGC uses about 745 MB. The naive result is still diagnostic only because correctness is not preserved on the broader layered benchmark, but the small run shows that the mild-recursive control is initially feasible for all engines.

At 100000, Morph-KGC completes `mild_recursive_context`, but with a much larger peak memory than SDM-RDFizer-Plus: approximately 23.27 GB for Morph-KGC versus 1.95 GB for SDM-RDFizer-Plus. It also completes L4C1, again with a much larger peak memory than SDM-RDFizer-Plus: approximately 23.7 GB for Morph-KGC versus 2.45 GB for SDM-RDFizer-Plus. Morph-KGC reaches the memory limit on L4C2 and L4C3. SDM-RDFizer-Plus completes `mild_recursive_context` and L4C2 at this size, but it reaches the memory limit on L4C3. The L4C3 kill is important: the unified implementation improves the range of completed deep cases, but it does not remove the worst-case memory pressure of the strongest cross-source alignment pattern.

At 500000, Morph-KGC reaches the memory limit on `mild_recursive_context`, L4C1, L4C2, and L4C3. For `mild_recursive_context`, the reported Morph-KGC time of about 160.97 seconds is therefore only a time-to-kill value, and the memory value marks the configured boundary rather than a successful peak memory. In the same configuration, SDM-RDFizer-Plus completes the run in about 1417.85 seconds with about 11.97 GB peak RSS. SDM-RDFizer-Plus also completes L4C1 at 500000, but reaches the memory limit on L4C2 and L4C3. In other words, SDM-RDFizer-Plus pushes the feasible boundary further for some reuse-heavy and deep mappings, but the largest recursive/alignment-style cases still exceed the configured memory budget.

L4C1 shows how rapidly the deep-case memory pressure appears while still allowing a meaningful comparison. From 10000 to 100000, Morph-KGC grows from about 768 MB to about 23.7 GB, already a roughly 30.9x increase for a 10x input increase.

SDM-RDFizer-Plus grows from about 187 MB to about 2.45 GB over the same step, a roughly 13.1x increase. The later SDM-RDFizer-Plus step from 100000 to 500000 is more regular: the input grows by 5x and memory grows from about 2.45 GB to about 13.2 GB, which is close to linear scaling. Morph-KGC, however, is already memory-killed on L4C1 at 500000. This makes L4C1 a clearer illustration of where the engines diverge: the pressure is already visible by 100000, and the larger run separates the feasible and infeasible execution ranges of the two engines.

Table 6.1: Growth factors for L4C1 from 10000 to 500000.

Engine	Time factor	Memory factor	500K status	Interpretation
Naive SDM-RDFizer	43.02x	6.17x	completed	Not a fair performance baseline because correctness is not preserved.
Morph-KGC	10.91x	37.21x	killed	Time is truncated; memory reaches the configured limit.
SDM-RDFizer-Plus	205.83x	70.85x	completed	Completes the largest run, but retained quoted payloads make the full 10K–500K growth substantial.

The likely explanation is structural. Morph-KGC’s partition execution is very effective when mapping rules can be grouped into efficient partitions, but the deep RDF-star patterns in this benchmark create several quoted contexts over the same underlying rows. When these contexts are combined with cross-source joins, the engine may need to hold large intermediate dataframe views, merged join results, or repeated contextualized columns. This is especially costly in patterns where both subject and object positions are quoted through joins. For SDM-RDFizer-Plus, the pressure comes from a different place: the unified cache, equivalence cache, quoted PJTT, and equivalence join table deliberately retain intermediate quoted payloads to avoid omission and order-sensitive reuse. That retention is what makes the output complete, but in L4C2 and L4C3 at 500000 it also becomes large enough to hit the memory limit.

6.6 Discussion

The results support three main conclusions. First, correctness must precede performance interpretation. The naive SDM-RDFizer implementation cannot be used as a fair time or memory baseline for these layered RML-star mappings because its lower resource use is entangled with omitted, malformed, or failed output. Second, Morph-KGC is strong on runtime for supported level-2 and level-3 patterns, which is consistent with its partition-based materialization design, but the deepest quoted-context patterns expose substantial memory pressure and some failure modes. Third, SDM-RDFizer-Plus improves robustness by explicitly retaining and reusing quoted payloads, and this allows it to complete several reuse-heavy and deep cases that Morph-KGC cannot complete under the evaluated memory boundary.

The trade-off is that robustness is not free. The unified cache, equivalence-aware cache, quoted PJTT, and equivalence join table preserve the intermediate payloads needed for complete RDF-star materialization. On shallow and medium-depth configurations, this cost is controlled and sometimes lower than Morph-KGC’s dataframe-oriented overhead. On the deepest configurations, especially L4C2 and L4C3, retained quoted payloads and join views grow large enough to dominate memory. This is the main remaining optimization target for future work: the unified implementation has solved the completeness problem for the benchmark family, but its cache lifetime and payload representation still need to be tightened for the largest cross-source recursive patterns.

Chapter 7

Conclusions and Future Work

7.1 Conclusion

This thesis addressed the problem of robust RML-star materialization in SDM-RDFizer-style execution. The starting point was the observation that a naive extension of an efficient RML materializer can appear fast and memory-light on simple mappings, while still being incomplete for more complex RDF-star dependency patterns. In particular, layered mappings that reuse quoted statements across same-source and cross-source access paths require more than ordinary row-wise triple generation. They require explicit knowledge of which triples maps produce quoted intermediate results, which later mappings consume them, which support mappings are equivalent to already materialized statement slices, and how long these intermediate results must remain available.

The main contribution of the thesis is a unified execution strategy for SDM-RDFizer-Plus. The implementation preserves the two-phase structure of SDM-RDFizer, namely Triples Map Planning followed by Triples Map Execution, but extends both phases with RML-star-specific mechanisms. In the planning phase, the unified approach introduces dependency analysis for quoted producers and consumers, statement-level equivalence detection, and dependency-aware flush boundaries. In the execution phase, it adds row-keyed and join-keyed reuse structures for quoted payloads, including the unified cache, the equivalence-aware cache, the quoted PJTT view, and the equivalence join table. Together, these mechanisms allow the materializer to reuse quoted results without relying on order-sensitive recursive rematerialization.

The evaluation showed why this additional machinery is necessary. The naive SDM-RDFizer implementation cannot be used as a fair runtime or memory baseline

for the layered RML-star benchmark, because it does not reliably materialize the complete expected output. Its lower resource usage in some cases is therefore not evidence of better efficiency, but partly a consequence of omitted or failed work. Morph-KGC provides a stronger external comparison. On supported level-2 and level-3 mappings, Morph-KGC shows strong runtime behavior, which is consistent with its partition-based execution design. However, on deeper quoted-context mappings it reaches the memory limit in several cases, and one large mild-recursive run is killed at the configured memory limit rather than completing materialization.

For this reason, memory efficiency in this thesis must be interpreted as memory efficiency for a correct materialization task. Under that interpretation, SDM-RDFizer-Plus is more meaningful than the naive SDM-RDFizer baseline even when the naive engine reports lower peak memory. The naive engine may use less memory because it omits quoted layers or fails before the complete result is produced. SDM-RDFizer-Plus spends additional memory on the unified cache, equivalence cache, and quoted join views, but this memory is used to preserve the RDF-star dependencies that the naive execution does not reliably preserve. On the level-2 and level-3 configurations, this correctness-preserving memory cost remains controlled, and at 100000 rows it is even lower than Morph-KGC for all supported configurations in that range.

SDM-RDFizer-Plus improves the robustness boundary. It completes the full 31line correctness gate for all layered configurations and completes several deep scaling cases that Morph-KGC cannot complete under the same memory limit. This confirms the central claim of the thesis: explicit star-aware planning, managed quoted-result reuse, and dependency-aware cache lifetime make the materializer more complete and less order-sensitive for complex RML-star mappings.

The correctness comparison is therefore also two-sided. Compared with the naive SDM-RDFizer implementation, SDM-RDFizer-Plus avoids the missing-layer behavior that makes the baseline unsuitable for the layered benchmark. Compared with Morph-KGC, SDM-RDFizer-Plus preserves the full correctness gate, including the strongest alignment-style configuration, while Morph-KGC is correct on the supported cases but exposes a limitation on the most complex quoted subject/object join combination. This does not diminish Morph-KGC’s runtime strengths on supported mappings, but it shows that the unified SDM-RDFizer-Plus execution model covers a broader part of the RML-star dependency space considered in this thesis.

At the same time, the evaluation also identifies the main remaining limitation. Correct quoted-result reuse is not free. The unified implementation retains intermediate quoted payloads so that later consumers can reuse them safely. This retention is what prevents omission, but it also increases memory pressure in the largest recursive and cross-source alignment patterns. In particular, the hardest level-4 con-

figurations show that the current cache representation can still grow large enough to reach the memory limit. The result is therefore a trade-off: SDM-RDFizer-Plus improves correctness and robustness substantially, but future work must reduce the memory footprint of the retained quoted payloads.

7.2 Future Work

The most important direction for future work is memory reduction for quoted intermediate results. The current implementation stores reusable quoted payloads in a direct form that is convenient for lookup and reuse, but not necessarily compact. As the evaluation showed, this representation becomes expensive when many rows contribute to multiple quoted contexts or when the same statement-level structure is reached through several join paths.

A first improvement would be to introduce compact payload representations. For example, instead of retaining complete quoted statement strings or full row-level payload objects, the engine could store dictionary identifiers, statement-template identifiers, and small arrays of row-dependent term IDs. The quoted statement could then be reconstructed only when it is actually emitted or passed to a later consumer. This would turn many repeated textual or object-level payloads into compact references.

A second direction is structural compression of equivalent payloads. The QET already identifies cases where a non-asserted support triples map and an owner-side predicate-object-map slice denote the same statement-level payload. Future work could use this equivalence information more aggressively by storing one canonical compressed payload per equivalence group and representing other members as aliases. This would reduce duplication between the unified cache and the equivalence-aware cache, especially in mappings where the same quoted statement is reused through both same-row and join-based access.

A third direction is join-view compression. The quoted PJTT and equivalence join table currently expose row tokens for later lookup. In large datasets, these token lists can become large, especially when many rows share the same join value. Future implementations could use compressed integer arrays, sorted delta encoding, bitmap-style representations, or chunked row-token blocks to reduce the memory footprint of these join-keyed structures without changing their logical behavior.

Finally, the benchmark itself can be extended. The layered mappings used in this thesis deliberately stress quoted-result reuse, equivalence, and cross-source joins. Future evaluations should add more real-world RML-star mappings, additional source

formats, and larger machines with different memory profiles. This would make it possible to separate implementation-specific memory overhead from the inherent cost of materializing deeply nested RDF-star structures. Overall, the next step is not to weaken the correctness guarantees introduced by the unified strategy, but to make those guarantees cheaper. The most promising path is to keep the same dependency-aware execution model while compressing, canonicalizing, and selectively materializing the quoted intermediate payloads that currently dominate memory in the hardest cases.

Bibliography

- [1] *Any plans to support rdf-star / sparql-star?* Ontop Google Group discussion thread. Accessed 2026-01-06. URL: <https://groups.google.com/g/ontop4obda/c/3vxN0hIux68>.
- [2] Julián Arenas-Guerrero, David Chaves-Fraga, Jhon Toledo, María S. Pérez, and Oscar Corcho. “Morph-KGC: Scalable knowledge graph materialization with mapping partitions”. In: *Semantic Web* 15.1 (2024), pp. 1–20. DOI: [10.3233/SW-223135](https://doi.org/10.3233/SW-223135), eprint: <https://journals.sagepub.com/doi/pdf/10.3233/SW-223135>, URL: <https://journals.sagepub.com/doi/abs/10.3233/SW-223135>.
- [3] Julián Arenas-Guerrero, Ana Iglesias-Molina, David Chaves-Fraga, Daniel Garijo, Oscar Corcho, and Anastasia Dimou. “Declarative generation of RDF-star graphs from heterogeneous data”. In: *Semantic Web* 16 (Feb. 2025), pp. 1–18. DOI: [10.3233/SW-243602](https://doi.org/10.3233/SW-243602).
- [4] Julián Arenas-Guerrero, Mario Scrocca, Ana Iglesias-Molina, Jhon Toledo, Luis Pozo, Daniel Doña, Oscar Corcho, and David Chaves-Fraga. “Knowledge Graph Construction with R2RML and RML: An ETL System-based Overview”. In: June 2021.
- [5] Christian Bizer, Tom Heath, and Tim Berners-Lee. “Linked Data - The Story So Far”. In: *International Journal on Semantic Web and Information Systems (IJSWIS)* 5.3 (July 2009), pp. 1–22. URL: <https://ideas.repec.org/a/igg/jswis0/v5y2009i3p1-22.html>.
- [6] Diego Calvanese, Benjamin Cogrel, Sarah Komla-Ebri, Roman Kontchakov, Davide Lanti, Martín Rezk, Mariano Rodriguez-Muro, and Guohui Xiao. “Ontop: Answering SPARQL queries over relational databases”. In: *Semantic Web* 8 (2016), pp. 471–487. URL: <https://api.semanticscholar.org/CorpusID:7721653>.
- [7] Diego Calvanese, Giuseppe De Giacomo, and Maurizio Lenzerini. “Data Integration: A Logic-Based Perspective”. In: *AI Magazine* 26.1 (2005), pp. 59–79. URL: <https://www.diag.uniroma1.it/~degiacono/papers/2005/calv-degi-AI-mag-2005.pdf>.
- [8] CARML Community. *CARML: An RML Mapping Engine*. <https://github.com/carml/carml>. Accessed: 2025-09-11. 2023.
- [9] Enrico Daga, Luigi Asprino, Paul Mulholland, and Aldo Gangemi. “Facade-X: an opinionated approach to SPARQL anything”. In: *Studies on the Semantic Web*. Vol. 53. 2021, pp. 58–73. DOI: [10.3233/SSW210035](https://doi.org/10.3233/SSW210035).

- [10] de Vleeschauwer, Els and Van Assche, Dylan and De Meester, Ben. “RMLMapper supported by RML-view-to-CSV in the KGCW Challenge 2025”. eng. In: *KGCW 2025 : Knowledge Graph Construction*. Ed. by Chaves-Fraga, David and Dasoulas, Ioannis and Debruyne, Christophe and Dimou, Anastasia and Serles, Umutcan and Van Assche, Dylan. Vol. 3999. Portoroz, Slovenia: CEUR, 2025, p. 3.
- [11] Thomas Delva, Julián Arenas Guerrero, Ana Iglesias Molina, Oscar Corcho, David Chaves Fraga, and Anastasia Dimou. “RML-star: A Declarative Mapping Language for RDF-star Generation”. In: *CEUR Workshop Proceedings 2980* (Oct. 2021). Ontology Engineering Group OEG. ISSN: 1613-0073. URL: <https://ceur-ws.org/Vol-2980/paper374.pdf>.
- [12] Anastasia Dimou, Tom De Nies, Ruben Verborgh, Erik Mannens, and Rik Van de Walle. “Automated Metadata Generation for Linked Data Generation and Publishing Workflows”. In: *Proceedings of the 9th Workshop on Linked Data on the Web*. Ed. by Søren Auer, Tim Berners-Lee, Christian Bizer, and Tom Heath. Vol. 1593. CEUR Workshop Proceedings. Apr. 2016.
- [13] Anastasia Dimou, Miel Vander Sande, Pieter Colpaert, Ruben Verborgh, Erik Mannens, and Rik Van de Walle. “RML: A Generic Language for Integrated RDF Mappings of Heterogeneous Data”. In: *Proceedings of the 7th Workshop on Linked Data on the Web*. Ed. by Christian Bizer, Tom Heath, Søren Auer, and Tim Berners-Lee. Vol. 1184. CEUR Workshop Proceedings. Apr. 2014. URL: http://ceur-ws.org/Vol-1184/ldow2014_paper_01.pdf.
- [14] Lisa Ehrlinger and Wolfram Wöß. “Towards a Definition of Knowledge Graphs”. In: Sept. 2016.
- [15] Gerald Haesendonck, Wouter Maroy, Pieter Heyvaert, Ruben Verborgh, and Anastasia Dimou. “Parallel RDF generation from heterogeneous big data”. In: *Proceedings of the International Workshop on Semantic Big Data*. July 2019. ISBN: 978-1-4503-6766-0. DOI: [10.1145/3323878.3325802](https://doi.org/10.1145/3323878.3325802) URL: <https://dl.acm.org/authorize?N680652>.
- [16] Gerald Haesendonck, Sitt Min Oo, Gertjan De Mulder, Michiel Derveeuw, Pieter Heyvaert, Wouter Maroy, Vincent Emonet, kmhaeren, Ben De Meester, Dylan Van Assche, Thomas, and ajuvercr. *RMLio/RMLStreamer: v2.5.0*. Software release. June 2023. DOI: [10.5281/zenodo.7998156](https://doi.org/10.5281/zenodo.7998156) URL: <https://zenodo.org/records/7998156>.
- [17] Alon Halevy, Anand Rajaraman, and Joann Ordille. “Data integration: the teenage years”. In: *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB '06)*. Seoul, Korea: VLDB Endowment, 2006, pp. 9–16. URL: <http://dl.acm.org/citation.cfm?id=1182635.1164130>.
- [18] Olaf Hartig. “RDF* and SPARQL*: An Alternative Approach to Annotate Statements in RDF”. In: *International Workshop on the Semantic Web*. 2017. URL: <https://api.semanticscholar.org/CorpusID:4534494>.
- [19] Pieter Heyvaert, Anastasia Dimou, Erik Mannens, David Chaves-Fraga, Freddy Priyatna, Oscar Corcho, and Ruben Verborgh. “Conformance Test Cases for the RDF Mapping Language (RML)”. In: *Proceedings of the 1st Iberoamerican Knowledge Graphs and Semantic Web Conference*. Ed. by Boris Villazón-Terrazas and Yusniel Hidalgo-Delgado. June 2019, pp. 162–173. ISBN: 978-3-030-21395-4. DOI: [10.1007/978-3-030-21395-4_12](https://doi.org/10.1007/978-3-030-21395-4_12).

-
- [20] Pieter Heyvaert, Dylan Van Assche, Ben De Meester, Gerald Haesendonck, Els de Vleeschauwer, and Sitt Min Oo. *RMLMapper-JAVA*. Zenodo (software). DOI: [10.5281/zenodo.3929132](https://doi.org/10.5281/zenodo.3929132). URL: <https://github.com/RMLio/rmlmapper-java>.
 - [21] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jose Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. “Knowledge Graphs”. In: *ACM Computing Surveys* 54 (July 2021), pp. 1–37. DOI: [10.1145/3447772](https://doi.org/10.1145/3447772).
 - [22] Enrique Iglesias. *SDM-RDFizer: An RML Interpreter for the Efficient Creation of RDF Knowledge Graphs*. Version 4.7.5.13.2. Open-source RML materialization engine with RML-star support. 2025. URL: <https://github.com/SDM-TIB/SDM-RDFizer>.
 - [23] Enrique Iglesias, David Chaves-Fraga, Jon De Las Heras, and Oscar Corcho. “SDM-RDFizer: An RML Interpreter for the Efficient Creation of RDF Knowledge Graphs”. In: *International Semantic Web Conference*. Springer, 2020, pp. 540–557. DOI: [10.1007/978-3-030-62466-8_34](https://doi.org/10.1007/978-3-030-62466-8_34).
 - [24] Enrique Antonio Iglesias and Maria-Esther Vidal. “Results for Knowledge Graph Creation Challenge 2024: SDM-RDFizer”. In: *Submitted to Fifth International Workshop on Knowledge Graph Construction@ESWC2024*. under review. 2024. URL: <https://openreview.net/forum?id=xyld7rVcV1>.
 - [25] Ana Iglesias-Molina, Julián Arenas-Guerrero, Thomas Delva, Anastasia Dimou, and David Chaves-Fraga, eds. *RML-star: Draft Community Group Report 10 May 2023*. W3C Knowledge Graph Construction Community Group report (editor’s draft). May 2023. URL: <https://w3id.org/rml/star/spec> (visited on 01/06/2026).
 - [26] Maurizio Lenzerini. “Data Integration: A Theoretical Perspective”. In: *Proceedings of the ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems*. June 2002, pp. 233–246. DOI: [10.1145/543613.543644](https://doi.org/10.1145/543613.543644).
 - [27] Frank Manola and Eric Miller. “RDF Primer”. In: (2004).
 - [28] *morph-kgc (PyPI package)*. <https://pypi.org/project/morph-kgc/>. Accessed 2026-01-07.
 - [29] *Morph-KGC Documentation*. <https://morph-kgc.readthedocs.io/en/latest/documentation/>. Accessed 2026-04-22.
 - [30] *Morph-KGC: Powerful RDF Knowledge Graph Generation with RML Mappings*. <https://github.com/morph-kgc/morph-kgc>. Accessed 2026-01-07.
 - [31] Boris Motik, Bernardo Cuenca Grau, Ian Horrocks, Zhe Wu, Achille Fokoue, and Carsten Lutz. *OWL 2 Web Ontology Language Profiles (Second Edition)*. W3C Recommendation. W3C, 2012. URL: <https://www.w3.org/TR/owl2-profiles/>.
 - [32] *Ontop Guide: Command Line Interface (Materialization)*. Official Ontop documentation. Accessed 2026-01-06. URL: <https://ontop-vkg.org/guide/cli.html>.
 - [33] *Ontop Guide: Introduction and Main Features*. Official Ontop documentation. Accessed 2026-01-06. URL: <https://ontop-vkg.org/guide/>.
 - [34] *Ontop Guide: Release Notes*. Official Ontop documentation. Accessed 2026-01-06. URL: <https://ontop-vkg.org/guide/releases.html>.

- [35] *ontop/ontop (rdfstar branch)*. GitHub branch. Accessed 2026-01-06. URL: <https://github.com/ontop/ontop/tree/rdfstar>.
- [36] *ontop/ontop: Ontop is a platform to query relational databases as Virtual RDF Knowledge Graphs using SPARQL*. GitHub repository. Accessed 2026-01-06. URL: <https://github.com/ontop/ontop>.
- [37] Sitt Min Oo, Gerald Haesendonck, Ben De Meester, and Anastasia Dimou. “RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data”. In: *The Semantic Web – ISWC 2022: 21st International Semantic Web Conference, Virtual Event, October 23–27, 2022, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2022, pp. 697–713. ISBN: 978-3-031-19432-0. DOI: [10.1007/978-3-031-19433-7_40](https://doi.org/10.1007/978-3-031-19433-7_40) URL: https://doi.org/10.1007/978-3-031-19433-7_40.
- [38] Heiko Paulheim. “Knowledge graph refinement: A survey of approaches and evaluation methods”. In: *Semantic Web 8.3* (2016), pp. 489–508. DOI: [10.3233/SW-160218](https://doi.org/10.3233/SW-160218). eprint: <https://journals.sagepub.com/doi/pdf/10.3233/SW-160218>. URL: <https://journals.sagepub.com/doi/abs/10.3233/SW-160218>.
- [39] RMLio contributors. *RMLStreamer*. GitHub repository. URL: <https://github.com/RMLio/RMLStreamer> (visited on 01/06/2026).
- [40] RMLio/RMLStreamer contributors. *Removing duplicates from the output? (Issue #26)*. GitHub issue. URL: <https://github.com/RMLio/RMLStreamer/issues/26> (visited on 01/06/2026).
- [41] semantify.it. *RocketRML*. GitHub repository. License: CC-BY-SA-4.0. Latest visible commit on default branch dated 2023-01-12. 2023. URL: <https://github.com/semantifyit/RocketRML> (visited on 01/06/2026).
- [42] Umutcan Şimşek, Elias Kärle, and Dieter Fensel. *RocketRML - A NodeJS implementation of a use-case specific RML mapper*. 2019. arXiv: [1903.04969 \[cs.SE\]](https://arxiv.org/abs/1903.04969). URL: <https://arxiv.org/abs/1903.04969>.
- [43] Lukas Sundqvist. “Extending VKG Systems with RDF-star Support”. In: 2022. URL: <https://api.semanticscholar.org/CorpusID:248883802>.
- [44] Maria-Esther Vidal. *Knowledge Graph Creation and Enrichment (Lecture 9)*. Lecture slides, Scientific Data Management and Knowledge Graphs, Leibniz University Hannover. Winter Semester 2024/25. 2024.
- [45] Maria-Esther Vidal. *Lecture 2*. Lecture slides, Scientific Data Management and Knowledge Graphs, Leibniz University Hannover. Accessed during the winter semester 2025. 2024.
- [46] W3C. *Data on the Web Best Practices*. <https://www.w3.org/TR/dwbp/>. W3C Recommendation. 2017.
- [47] W3C. *R2RML: RDB to RDF Mapping Language*. <https://www.w3.org/TR/r2rml/>. W3C Recommendation. 2012.
- [48] W3C. *RDF 1.1 Concepts and Abstract Syntax*. <https://www.w3.org/TR/rdf11-concepts/>. W3C Recommendation. 2014.

- [49] Guohui Xiao, Davide Lanti, Roman Kontchakov, Sarah Komla-Ebri, Elem Güzel-Kalaycı, Linfang Ding, Julien Corman, Benjamin Cogrel, Diego Calvanese, and Elena Botoeva. “The Virtual Knowledge Graph System Ontop”. In: *The Semantic Web – ISWC 2020*. Ed. by Jeff Z. Pan, Valentina Tamma, Claudia d’Amato, Krzysztof Janowicz, Bo Fu, Axel Polleres, Oshani Seneviratne, and Lalana Kagal. Cham: Springer International Publishing, 2020, pp. 259–277. ISBN: 978-3-030-62466-8.